

UNIVERZITET U BEOGRADU
ELEKTROTEHNIČKI FAKULTET



OPTIMALNA RASPODELA POSLOVA NA RAČUNARIMA

Master rad

Mentori:	Kandidat:
dr Jovana Petrović, docent dr Dragan Olćan, profesor	Mihailo Radojević 2023/3209

Beograd, jul 2026.

Sadržaj

1	Uvod	3
1.1	Opis problema	3
1.2	Matematička formulacija	4
1.3	Složenost problema i prostor pretrage	4
1.4	Nelinearno programiranje	5
1.5	Celobrojno linearno programiranje i softverski alat HiGHS	5
1.6	Stohastički algoritmi za probleme velikih dimenzija	6
1.7	Cilj rada i doprinosi	7
1.8	Struktura rada	7
2	Pretraga rešenja genetičkim algoritmom	8
2.1	Pregled algoritma	8
2.2	Reprezentacija i funkcija cilja	8
2.3	Koraci algoritma	8
2.4	Blok dijagram algoritma	9
2.5	Detalji operatora selekcije, ukrštanja i mutacije	10
2.6	Hiperparametri i njihovo podešavanje	11
2.7	Procedura popravke	11
2.8	Implementacija u programskim jezicima	12
2.9	Kriterijumi zaustavljanja	12
2.10	Hardverska konfiguracija eksperimenata	12
3	Rezultati	13
3.1	Postavka eksperimenta	13
3.2	Problem male dimenzije	13
3.2.1	Rešenje alatom HiGHS	13
3.2.2	Rešenje genetičkim algoritmom	13
3.2.3	Poređenje algoritama	14
3.2.4	Analiza kumulativnih maksimuma	15
3.3	Problem srednje dimenzije	18
3.3.1	Rešenje alatom HiGHS	18
3.3.2	Rešenje genetičkim algoritmom	19
3.3.3	Poređenje algoritama	19
3.3.4	Analiza kumulativnih maksimuma	20
3.4	Problem velike dimenzije	22
3.4.1	Rešenje alatom HiGHS	22
3.4.2	Rešenje genetičkim algoritmom	22
3.4.3	Poređenje algoritama	22
3.4.4	Analiza kumulativnih maksimuma	24
4	Analiza i diskusija rezultata	27
4.1	Uticaj hiperparametara	27
4.2	Poređenje sa slučajnom pretragom	29
4.3	Kvalitet početne populacije i doprinos procedure popravke	29
4.4	Analiza stabilnosti rezultata	30

4.5	Analiza vremenske složenosti	30
4.6	Konvergencioni profil i zaključci	31
4.7	Sažeta diskusija rezultata	31
5	Zaključak	33
5.1	Pregled doprinosa	33
5.2	Ograničenja predloženog pristupa	33
5.3	Pravci daljeg rada	34
5.4	Praktične preporuke	34
	Literatura	36
	Spisak skraćenica	37
	Spisak slika	38
	Spisak tabela	39

1 Uvod

Cilj ovog rada je da se ispita problem maksimizacije zarade pri raspodeli servisa na heterogeni skup računara i da se isti reši primenom genetičkog algoritma. U ovom poglavlju formalno je definisan zadatak, izvedena je njegova matematička formulacija i navedeni su pristupi rešavanju nelinearnim programiranjem (eng. *nonlinear programming*, NLP) i celobrojnim linearnim programiranjem (eng. *integer linear programming*, ILP). Kao alternativa za probleme velikih dimenzija uvedene su stohastičke metode. U drugom poglavlju opisana je pretraga rešenja genetičkim algoritmom kroz kratak pregled koraka algoritma sa pratećim blok dijagramom. U trećem poglavlju prikazani su rezultati za tri instance: problem male dimenzije (10×10), problem srednje dimenzije (100×100) i problem velike dimenzije (1000×1000), uključujući rešenja dobijena softverskim alatom HiGHS i drugim alatima za matematičku optimizaciju, rešenja genetičkog algoritma, poređenje sa slučajnom pretragom, poređenje vremena izvršavanja pri istoj hardverskoj konfiguraciji i analizu kumulativnih maksimuma kroz generacije. U četvrtom poglavlju data je detaljna analiza i diskusija rezultata, uključujući uticaj hiperparametara, doprinos procedure popravke, poređenje pri jednakom raspoloživom vremenu i analizu složenosti. Peto poglavlje sadrži zaključke i pravce daljeg rada.

1.1 Opis problema

Posmatra se sistem koji se sastoji od m tipova servisa i n heterogenih računara. Svaki servis predstavlja vrstu posla koji treba izvršiti, dok svaki računar predstavlja resurs sa ograničenim radnim vremenom. Za svaki par servis–računar poznato je vreme izvršavanja jedne jedinice servisa na tom računaru i zarada koja se ostvaruje po izvršenoj jedinici. Pored toga, svaki servis ima tržišni zahtev koji predstavlja broj jedinica koje je potrebno izvršiti u planiranom periodu. Svaki računar raspolaže fiksnim radnim vremenom, koje u eksperimentima sprovedenim u ovom radu iznosi $T = 2880$ minuta, što odgovara dva dana.

Zadatak je sledeći. Odrediti koliko jedinica svakog servisa rasporediti na svaki računar tako da ukupna zarada bude maksimalna, da ne bude prekoračeno vreme nijednog računara i da ne bude izvršeno više jedinica nijednog servisa nego što je zahtevano. Razlika u vremenima izvršavanja i zaradi po paru servis–računar potiče od činjenice da računari nisu identični. Različite mašine imaju različitu specijalizaciju, pa je profitabilnost izvršavanja istog servisa na različitim mašinama u opštem slučaju različita.

Problem ima jasnu poslovnu interpretaciju. U cloud računarstvu servisi odgovaraju zahtevima korisnika, a računari raznorodnim virtuelnim mašinama; zarada po jedinici tada predstavlja prihod koji provajder ostvaruje od jedne usluge, a vreme izvršavanja vreme zauzeća resursa. U industrijskoj proizvodnji servisi su artikli koji se izrađuju, a računari su proizvodne linije sa različitim performansama. U logistici servisi predstavljaju tipove paketa, a računari su distributivni centri sa različitim kapacitetima. U svakoj od navedenih primena veličina instance varira od desetina do nekoliko hiljada jedinica, a matematička struktura problema je ista, što opravdava razvoj generičkih metoda za rešavanje koje se efikasno skaliraju sa veličinom problema.

Iako je problem konceptualno jednostavan, broj kandidatskih rešenja eksponencijalno raste sa veličinom ulaza. Već za umerene veličine, recimo 100 servisa i 100 računara, prostor pretrage prevazilazi mogućnosti naivnih egzaktnih algoritama u prihvatljivom vremenu. Za još veće instance, reda hiljadu servisa i hiljadu računara, komercijalni softverski alati za matematičku optimizaciju mogu pronaći visokokvalitetna rešenja, ali im vreme i memorijski zahtevi značajno rastu. U takvim uslovima stohastičke i metaheurističke metode predstavljaju zanimljivu alternativu: za rešenje koje pronađu ne postoji

matematički dokaz optimalnosti, ali se zauzvrat dobijaju fleksibilnost modela i mogućnost upravljanja utrošenim vremenom.

1.2 Matematička formulacija

Uvedimo sledeće oznake. Neka je $S = \{1, \dots, m\}$ skup servisa, a $C = \{1, \dots, n\}$ skup računara. Neka je t_{ij} vreme izvršavanja jedne jedinice servisa $i \in S$ na računaru $j \in C$, p_{ij} zarada od te jedinice, d_i broj zahtevanih jedinica servisa i , a T maksimalno radno vreme svakog računara. Promenljive odlučivanja su celobrojne i nenegativne, $x_{ij} \in \mathbb{Z}_{\geq 0}$, gde x_{ij} predstavlja broj jedinica servisa i izvršenih na računaru j .

Problem se tada formuliše kao celobrojni linearni program:

$$\max \sum_{i=1}^m \sum_{j=1}^n p_{ij} x_{ij} \quad (1)$$

$$\text{pri ograničenjima} \quad \sum_{i=1}^m t_{ij} x_{ij} \leq T, \quad \forall j \in C \quad (2)$$

$$\sum_{j=1}^n x_{ij} \leq d_i, \quad \forall i \in S \quad (3)$$

$$x_{ij} \in \mathbb{Z}_{\geq 0}, \quad \forall i \in S, j \in C \quad (4)$$

Funkcija cilja (1) predstavlja ukupnu zaradu sumiranu kroz sve servise i sve računare. Ograničenje (2) izražava da ukupno vreme provedeno na računaru j ne sme premašiti T . Ograničenje (3) izražava da ukupan broj izvršenih jedinica servisa i na svim računarima ne sme premašiti zahtevani broj jedinica d_i . Uslov celobrojnosti (4) čini problem računski znatno težim od njegove kontinualne relaksacije.

1.3 Složenost problema i prostor pretrage

Razmatrani problem može se posmatrati kao generalizacija problema ranca [1]. Već za slučaj sa jednim računarem, problem se svodi na ograničeni problem ranca sa više predmeta, što je dobro poznat NP-težak problem [2]. U opštem slučaju, problem se može posmatrati kao varijanta generalizovanog problema dodeljivanja [3, 4] sa celobrojnim količinama umesto binarnih dodela. Direktna posledica složenosti je da i specijalizovane metode namenjene za specifične probleme imaju eksponencijalnu vremensku složenost u najgorem slučaju, što motiviše upotrebu stohastičkih i metaheurističkih metoda za velike instance.

Veličina prostora pretrage, ako se zanemare ograničenja, iznosi $\prod_{i,j} (1 + \min(d_i, \lfloor T/t_{ij} \rfloor))$, što za instancu 1000×1000 daje broj kandidatskih konfiguracija reda veličine 10^{2000} . Potpuna pretraga je očigledno neostvariva. Pored eksponencijalnog rasta broja kandidata, funkcija cilja je linearna, pa nema klasičnih lokalnih ekstrema u smislu diferencijalne analize, ali je zbog celobrojnih ograničenja prostor diskretan i ispunjen susednim rešenjima sa različitim vrednostima funkcije cilja. Mala promena jednog gena, na primer povećanje broja jedinica servisa i na računaru j za jedan, može zahtevati simultanu promenu više drugih gena kako bi rešenje zadovoljavalo sva ograničenja.

Ograničenja problema definišu poliedar dopustivih rešenja u prostoru realnih brojeva, dok celobrojnost zahteva da se konačno rešenje nalazi u tačkama rešetke unutar tog poliedra. Broj takvih tačaka je u opštem slučaju izuzetno veliki. Linearna programska relaksacija pomenutog poliedra najčešće

daje gornju granicu funkcije cilja koja se koristi u metodi grananja i ograničavanja, ali odstojanje između relaksiranog i celobrojnog optimuma može biti znatno za neke instance, što direktno utiče na efikasnost specijalizovanih metoda.

Sa stanovišta metaheurističkih metoda, ovakva struktura prostora pretrage ima dvostruki uticaj. Sa jedne strane, rešenje se može direktno predstaviti matricom celih brojeva, bez posebnog kodiranja i dekodiranja hromozoma. Sa druge strane, narušavanje dopustivosti kod gotovo svake izmene zahteva ugradnju mehanizma popravke koji ne samo da vraća rešenje u dopustiv region, već ponekad i popravlja vrednost funkcije cilja. Bez takvog mehanizma, populacija bi se zagušila nevažećim hromozomima i evolucija ne bi mogla da napreduje.

1.4 Nelinearno programiranje

Iako je razmatrani problem prirodno linearan po promenljivima x_{ij} kada su koeficijenti t_{ij} , p_{ij} i d_i unapred poznati, u realnim primenama često se sreću proširenja koja uvode nelinearnosti. Tipičan primer je situacija u kojoj zarada po jedinici servisa nije konstantna, već zavisi od ukupnog broja izvršenih jedinica, na primer kroz količinske popuste ili efekat zasićenja tržišta. U tim slučajevima funkcija cilja postaje konkavna ili nekonveksna, a problem prelazi u domen nelinearnog programiranja.

Druga klasa nelinearnih proširenja odnosi se na model trošenja vremena na računaru. Ako se vreme izvršavanja menja sa opterećenjem mašine, na primer zbog upravljanja energetskom potrošnjom ili nelinearnog ponašanja keša, ograničenje (2) prestaje da bude linearno. U opštem slučaju, NLP problemi nisu garantovano rešivi u polinomskom vremenu, čak ni za umerene veličine instanci. Lokalni optimumi mogu biti udaljeni od globalnog optimuma, a metode zasnovane na gradijentima zahtevaju glatkost funkcija koja ne mora biti zadovoljena.

U razmatranom radu pretpostavlja se linearni model, što je opravdano kako sa stanovišta poređenja različitih metoda rešavanja, tako i sa stanovišta primenljivosti rezultata. Linearni model takođe predstavlja prirodnu osnovu za kasnija proširenja na nelinearne varijante, jer rešenje linearnog problema može poslužiti kao početna inicijalizacija za iterativne NLP metode.

1.5 Celobrojno linearno programiranje i softverski alat HiGHS

Celobrojno linearno programiranje je grana matematičkog programiranja koja se bavi linearnim funkcijama cilja i ograničenjima uz dodatne uslove celobrojnosti promenljivih. Iako je linearno programiranje u kontinualnoj relaksaciji polinomski rešivo simpleks metodom ili metodom unutrašnje tačke [5], dodavanje uslova celobrojnosti čini problem NP-teškim. Softverski alati za matematičku optimizaciju (eng. *mathematical optimization software tools*) koji egzaktno rešavaju ILP probleme zasnovani su na metodama grananja i ograničavanja (eng. *branch-and-bound*), grananja i odsecanja (eng. *branch-and-cut*) i drugim sofisticiranim tehnikama koje eksploatišu strukturu linearne relaksacije za eliminaciju velikih delova prostora pretrage [6].

Među dostupnim open-source alatima, HiGHS [7, 8] se izdvaja kao jedan od najefikasnijih za probleme mešovitog celobrojnog linearnog programiranja. HiGHS implementira napredne tehnike pretprocesiranja, sečenje ravni (eng. *cutting planes*) i heuristike za pronalaženje dopustivih rešenja. Njegova prednost je dostupnost preko otvorenih licenci, što omogućava akademsku i industrijsku upotrebu bez restrikcija. U ovom radu HiGHS je korišćen kao primarni egzaktni alat za sve tri veličine instanci.

Pored HiGHS-a, postoje i komercijalni alati, među kojima se ističu Gurobi [9], CPLEX i Mosek. Ovi alati često postižu bolja vremena izvršavanja za izuzetno velike instance, ali zahtevaju plaćene

licence. U ovom radu Gurobi je korišćen preko NEOS servera radi potvrde referentnih vrednosti dobijenih HiGHS-om. Razlika između rezultata HiGHS-a i Gurobija je u svim ispitanim instancama bila zanemarljiva, što potvrđuje pouzdanost izabranog open-source alata.

Model ILP-a u ovom radu implementiran je u programskom jeziku Python uz pomoć biblioteke PuLP [10], koja služi kao apstrakcioni sloj iznad različitih alata za optimizaciju. Gornje granice celobrojnih promenljivih unapred su sužene na $\min(d_i, \lfloor T/t_{ij} \rfloor)$, čime se značajno smanjuje veličina linearne relaksacije i ubrzava postupak grananja i ograničavanja.

1.6 Stohastički algoritmi za probleme velikih dimenzija

Egzaktne metode zasnovane na ILP-u efikasno rešavaju male i srednje instance, ali se za veoma velike instance suočavaju sa ozbiljnim vremenskim i memorijskim ograničenjima. Linearna relaksacija problema sa milion promenljivih i nekoliko hiljada ograničenja zahteva značajne resurse, a faza grananja i ograničavanja se može znatno usporiti u slučajevima sa razgranatim drvetom pretrage. Pored toga, u dinamičkim scenarijima u kojima se ulazni podaci često menjaju, ponovno rešavanje punog ILP modela posle svake promene ulaza nije ekonomski opravdano.

Kao odgovor na ove izazove razvijene su stohastičke i metaheurističke metode. Karakteristika ovih metoda je da koriste slučajnost kao mehanizam za istraživanje prostora rešenja i da po prirodi nisu garantovano optimalne, ali za mnoge probleme daju visokokvalitetna rešenja u kraćem vremenu od egzaktnih metoda na velikim instancama. Da li se ta prednost ostvaruje zavisi od konkretnog problema i instance, što je jedno od pitanja koja ovaj rad eksperimentalno ispituje. Stohastičke metode su posebno korisne kada egzaktni pristup nije primenljiv ili kada se rešenja iz prethodnih instanci mogu iskoristiti kao početna inicijalizacija.

Najjednostavnija stohastička metoda je slučajna pretraga (eng. *random search*), koja generiše N slučajnih dopustivih rešenja i vraća najbolje. U ovom radu svako slučajno generisano rešenje prolazi kroz istu proceduru popravke kao i rešenja genetičkog algoritma (odjeljak 2.7), čime postaje dopustivo i lokalno popravljeno; taj korak treba imati u vidu pri tumačenju rezultata, jer deo kvaliteta slučajne pretrage potiče od same popravke. Iako trivijalna, ova metoda služi kao važna kontrolna grupa za poređenje sa složenijim algoritmima. Rezultat složenijeg algoritma može se smatrati značajnim samo ako jasno nadmašuje slučajnu pretragu pri istom broju evaluacija funkcije cilja.

Sofisticiranije stohastičke metode obuhvataju simulirano kaljenje (eng. *simulated annealing*), tabu pretragu, kolonije mrava (eng. *ant colony optimization*), optimizaciju jatom (eng. *particle swarm optimization*) i evolutivne algoritme. Genetički algoritmi [11, 12, 13] su među najpopularnijim predstavnicima evolutivnih algoritama, sa širokom primenom u rasporedu, planiranju, kombinatornom dizajnu i mašinskom učenju. Njihova ključna prednost je sposobnost istovremenog istraživanja više regiona prostora rešenja, što ih čini otpornim na lokalne ekstremume.

U ovom radu, kao predstavnik stohastičkih metoda za rešavanje razmatranog problema, izabran je genetički algoritam. Njegova primenljivost za razmatrani problem opravdana je sledećim razlozima. Prvo, prostor pretrage je diskretan i visokodimenzionalan, što odgovara osnovnoj postavci genetičkih algoritama. Drugo, rešenja koja narušavaju ograničenja mogu se specijalizovanom procedurom popravke [14] efikasno vratiti u dopustivu oblast, čime se izbegava korišćenje funkcija kazne koje često komplikuju podešavanje. Treće, funkcija cilja je separabilna po članovima, što olakšava analizu uticaja pojedinačnih gena.

1.7 Cilj rada i doprinosi

Cilj ovog rada je da se za navedeni problem maksimizacije zarade formuliše genetički algoritam koji efikasno pronalazi visokokvalitetna rešenja i da se njegove performanse uporede sa egzaktnim alatom HiGHS i slučajnom pretragom. Pored toga, cilj je i da se eksperimentalno odredi granica primenljivosti egzaktnog pristupa i da se identifikuju situacije u kojima stohastička metaheuristika postaje neophodna ili poželjna.

Konkretni doprinosi rada su sledeći. Najpre, predložena je formulacija problema kao varijanta generalizovanog problema dodeljivanja sa celobrojnim odlukama. Zatim je implementiran genetički algoritam sa specijalizovanom procedurom popravke koja integriše pohlepne lokalne heuristike i garantuje dopustivost svakog kandidatskog rešenja. Za selekciju su korišćeni turnirska selekcija i elitiizam, a pri dužoj stagnaciji najbolje vrednosti deo najlošijih hromozoma zamenjuje se novim slučajnim jedinkama (injekcija raznolikosti), čime se predupređuje prerana konvergencija. Implementacija je realizovana u dva programska jezika, Python i C++, kako bi se obezbedila čitljivost referentne verzije i performansa na velikim instancama. Najzad, sprovedena je sistematska eksperimentalna evaluacija na tri reprezentativne veličine problema, sa više pokretanja po konfiguraciji, što obezbeđuje statističku stabilnost zaključaka.

1.8 Struktura rada

U narednom poglavlju 2 predstavljen je genetički algoritam kao stohastička pretraga rešenja, sa kratkim opisom svakog koraka i pratećim blok dijagramom. U poglavlju 3 prikazani su eksperimentalni rezultati za tri veličine instanci: problem male dimenzije (10×10), problem srednje dimenzije (100×100) i problem velike dimenzije (1000×1000). U poglavlju 4 data je detaljna analiza i diskusija rezultata. U poglavlju 5 izvedeni su zaključci i navedeni pravci za dalji rad.

2 Pretraga rešenja genetičkim algoritmom

2.1 Pregled algoritma

Genetički algoritam je populaciono zasnovana stohastička metaheuristika koja održava skup kandidatskih rešenja, takozvanu populaciju, i kroz generacije ga usavršava primenom operatora selekcije, ukrštanja i mutacije. Inspiracija dolazi iz biološke evolucije po Darwinovom principu prirodne selekcije, gde jedinke sa boljom prilagođenošću imaju veću verovatnoću da prenesu svoje karakteristike na naredne generacije. Genetički algoritmi su uvedeni radom Džona Holanda [11] sedamdesetih godina prošlog veka, a popularizovani su kroz monografije Goldberga [12] i Ajbena i Smita [13]. Pregled genetičkog algoritma, sa strategijama selekcije, ukrštanja i mutacije, dat je i u materijalima sa predavanja predmeta Optimizacioni algoritmi u inženjerstvu [15].

Verzija genetičkog algoritma predstavljena u ovom radu sadrži dva specijalizovana mehanizma. Prvi je procedura popravke hromozoma, koja svako rešenje vraća u oblast dopustivosti i pritom ga lokalno poboljšava primenom pohlepnih heuristika. Drugi je injekcija raznolikosti, koja pri dužoj stagnaciji najbolje vrednosti zamenjuje deo najlošijih hromozoma novim slučajnim jedinkama i time populaciji vraća istraživački potencijal. Pored toga, u okviru operatora selekcije korišćen je elitizam, koji garantuje da se najbolja rešenja iz tekuće generacije neizmenjeno prenose u sledeću.

2.2 Reprezentacija i funkcija cilja

Hromozom je predstavljen matricom $X \in \mathbb{Z}_{\geq 0}^{m \times n}$ koja direktno opisuje broj jedinica svakog servisa na svakom računaru. Genotip i fenotip su identični, što izbegava potrebu za dekodiranjem koje bi inače bilo prisutno kod indirektnih reprezentacija. Cena ovog izbora je obaveza aktivne kontrole dopustivosti, koja se postiže pozivom procedure popravke posle svake izmene hromozoma.

Kao mera kvaliteta hromozoma koristi se direktno funkcija cilja problema:

$$f(X) = \sum_{i=1}^m \sum_{j=1}^n p_{ij} x_{ij}.$$

Pošto svaki hromozom posle popravke uvek zadovoljava ograničenja problema, dodatne kazne (eng. *penalty terms*) nisu potrebne, što značajno pojednostavljuje podešavanje algoritma.

2.3 Koraci algoritma

Algoritam se odvija kroz sledeće osnovne korake.

1. Inicijalizacija populacije. Generiše se P slučajnih hromozoma. Svaka vrednost x_{ij} uniformno se uzima iz intervala $[0, \min(d_i, \lfloor T/t_{ij} \rfloor)]$. Svaki hromozom potom prolazi kroz proceduru popravke, čime postaje istovremeno dopustiv i lokalno popravljen.

2. Izračunavanje funkcije cilja. Za svaki hromozom u populaciji izračunava se vrednost funkcije cilja $f(X)$.

3. Elitizam. Najboljih E hromozoma se kopira u sledeću generaciju bez izmena. Time se garantuje da najbolja vrednost funkcije cilja monotono ne opada kroz generacije.

4. Selekcija roditelja. Roditelji se biraju turnirskom selekcijom sa veličinom turnira k . Iz populacije se nasumično bira k hromozoma, a roditelj postaje hromozom koji pobeđi u turniru, odnosno onaj sa najvećom vrednošću funkcije cilja među izabranima. Turnirska selekcija ne zahteva normalizaciju vrednosti funkcije cilja, a selekcionni pritisak se neposredno kontroliše veličinom turnira k .

5. Ukrštanje. Sa verovatnoćom $p_c = 0,8$ primenjuje se jednotačkasto ukrštanje. Matrice roditelja se ravnaju u jednodimenzionalni niz dužine mn , bira se slučajna tačka preseka, i deca nastaju razmenom prefiksa i sufiksa. U suprotnom, deca su direktne kopije roditelja.

6. Mutacija. Sa verovatnoćom $p_m = 0,15$ hromozom se mutira tako što se bira L slučajnih gena (i, j) i vrednost svakog od njih zamenjuje se slučajnim brojem iz intervala $[0, \lfloor T/t_{ij} \rfloor]$. Broj istovremeno mutiranih gena L skalira se sa dimenzijom problema: u referentnoj Python implementaciji mutira se jedan gen, dok se u C++ implementaciji za srednje i velike instance mutira grupa od 50 gena, jer bi promena jednog gena od njih 10^4 do 10^6 imala zanemarljiv efekat. Relativno visoka stopa mutacije podnošljiva je zato što procedura popravke brzo otklanja destruktivne efekte.

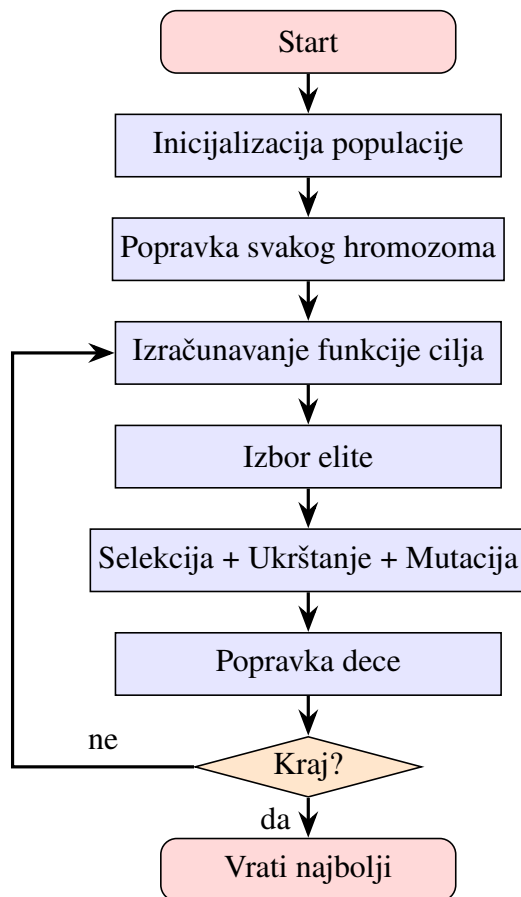
7. Popravka. Svako dete prolazi kroz proceduru popravke. Procedura ima četiri faze: odsecanje negativnih vrednosti, primena ograničenja zahteva proporcionalnim smanjivanjem redova, primena ograničenja vremena pohlepnim uklanjanjem najmanje profitabilnih jedinica iz kolona, i popunjavanje preostalog vremena pohlepnim dodavanjem najprofitabilnijih jedinica.

8. Zamena populacije. Nova generacija se formira od elitne grupe i novonastale dece. Ako populacija stagnira preko 100 generacija, aktivira se injekcija raznolikosti i 20% najlošijih hromozoma se zamenjuje slučajnim hromozomima.

9. Kriterijum zaustavljanja. Koraci 2 do 8 ponavljaju se sve dok se ne dostigne unapred zadati broj generacija G . Pored toga, prati se najbolji ikad pronađeni hromozom, koji se vraća kao konačno rešenje.

2.4 Blok dijagram algoritma

Slika 1 prikazuje blok dijagram glavne petlje genetičkog algoritma. Centralna petlja obuhvata korake od 2 do 7, dok se inicijalizacija izvršava jednom pre ulaska u petlju. Procedura popravke je eksplicitno prikazana kao izdvojeni blok pošto se poziva više puta po generaciji.



Slika 1: Blok dijagram genetičkog algoritma sa procedurom popravke i elitizmom.

2.5 Detalji operatora selekcije, ukrštanja i mutacije

Turnirska selekcija odvija se tako što se iz populacije nasumično, sa ponavljanjem, izabere k hromozoma, a za roditelja se proglašava onaj sa najvećom vrednošću funkcije cilja. Veličina turnira k direktno kontroliše selekcionni pritisak. Za $k = 1$ selekcija je potpuno slučajna i ne postoji pritisak ka boljim rešenjima, dok za k jednako veličini populacije uvek pobeđuje globalno najbolji hromozom, čime se raznolikost brzo gubi. Kod manjih populacija vrednosti k između 3 i 7 daju umeren pritisak koji favorizuje kvalitetne hromozome, ali povremeno propušta i slabije jedinke u narednu generaciju, čime se očuva raznolikost. Pošto selekcionni pritisak zavisi od odnosa k i veličine populacije, kod velikih populacija k se srazmerno podiže, do vrednosti reda 10 do 20.

Operator jednotačkastog ukrštanja radi nad linearizovanom reprezentacijom hromozoma. Matrica $X \in \mathbb{Z}_{\geq 0}^{m \times n}$ se ravna u niz dužine mn čitanjem po redovima, bira se slučajna tačka preseka $c \in \{1, \dots, mn - 1\}$, i prvo dete nasleđuje prefiks dužine c od prvog roditelja i sufiks od drugog, dok drugo dete nasleđuje komplementarne delove. Na primer, za hromozome dimenzije 2×2 linearizovane kao $[x_{11}, x_{12}, x_{21}, x_{22}]$, sa roditeljima $A = [3, 1, 0, 2]$ i $B = [0, 4, 5, 1]$ i tačkom preseka $c = 2$, dobijaju se deca $[3, 1, 5, 1]$ i $[0, 4, 0, 2]$. Pošto razmena prefiksa i sufiksa gotovo uvek narušava ograničenja vremena i zahteva, oba deteta odmah prolaze kroz proceduru popravke.

Operator mutacije bira L slučajnih gena (i, j) i vrednost svakog zamenjuje slučajnim celim brojem iz intervala $[0, \lfloor T/t_{ij} \rfloor]$. Za razliku od malih perturbacija koje samo blago menjaju postojeću vrednost, ovakva potpuna zamena omogućava velike skokove u prostoru pretrage i pomaže izlasku iz lokalnih

optimuma. Relativno visoka stopa mutacije $p_m = 0,15$ podnošljiva je upravo zato što procedura popravke deluje kao stabilizator: ona popravljajući destruktivne efekte mutacije i istovremeno lokalno poboljšava rešenje, pa se agresivnija eksploracija ne plaća gubitkom u konvergenciji.

2.6 Hiperparametri i njihovo podešavanje

Genetički algoritam zavisi od nekoliko hiperparametara čije vrednosti značajno utiču na kvalitet rešenja. Veličina populacije P određuje broj kandidatskih rešenja u svakoj generaciji. Broj generacija G određuje ukupan broj iteracija evolucije. Veličina elitne grupe E određuje koliko se najboljih hromozoma direktno prenosi u sledeću generaciju. Verovatnoća ukrštanja p_c kontroliše učestalost rekombinacije, a verovatnoća mutacije p_m učestalost stohastičkih modifikacija. Veličina turnira k određuje selekcionarni pritisak.

Eksperimenti sprovedeni u ovom radu pokazuju da vrednosti $p_c = 0,8$ i $p_m = 0,15$ daju robusne rezultate na svim ispitanim veličinama instanci, dok se veličina turnira k skalira sa populacijom, od k između 3 i 7 kod malih do k između 10 i 20 kod velikih populacija. Veličina elite oko 3% do 5% populacije pokazala se kao dobar kompromis između čuvanja kvalitetnih rešenja i sprečavanja prerane konvergencije.

2.7 Procedura popravke

Procedura popravke se izvršava nakon svake izmene hromozoma i obavlja se u četiri koraka.

Prvi korak je odsecanje negativnih vrednosti. Sve vrednosti hromozoma manje od nule postaju nula. Ovaj korak je neophodan jer pojedini operatori mogu privremeno proizvesti negativne vrednosti.

Drugi korak je primena ograničenja zahteva po redovima. Ako je suma vrednosti u nekom redu veća od zahteva za odgovarajući servis, sve vrednosti tog reda se proporcionalno smanjuju faktorom $d_i / \sum_j x_{ij}$, čime se očuvava relativna raspodela jedinica po računarima.

Treći korak je primena ograničenja vremena po kolonama. Ako za neku kolonu ukupno vreme prelazi T , jedinice se uklanjaju iz onih servisa koji imaju najnižu profitabilnost po minutu, odnosno najmanji odnos p_{ij}/t_{ij} , sve dok se vremensko ograničenje ne ispuni.

Četvrti korak je popunjavanje preostalog vremena. Za svaku kolonu, dok god ima slobodnog vremena i dok zahtev nije iscrpljen, dodaju se jedinice servisa po opadajućem p_{ij}/t_{ij} . Ovaj korak ima pohlepni karakter i značajno povećava kvalitet rešenja.

Treći i četvrti korak predstavljaju pohlepne lokalne heuristike koje povećavaju kvalitet rešenja bez narušavanja dopustivosti. Time je svaki potomak istovremeno i dopustiv i lokalno-poboljšan, što znatno ubrzava konvergenciju.

Redosled koraka je bitan: odsecanje negativnih vrednosti mora prethoditi ostalim koracima jer bi negativne vrednosti pokvarile sume po redovima i kolonama, a primena ograničenja zahteva dolazi pre ograničenja vremena jer smanjenje po redovima može osloboditi vreme na računarima i tako smanjiti obim posla u trećem koraku. Četvrti korak, pohlepno popunjavanje, uvek dolazi poslednji jer pretpostavlja da su sva ograničenja već zadovoljena, pa preostalo slobodno vreme popunjava najprofitabilnijim jedinicama.

Sortiranje servisa po odnosu profitabilnosti p_{ij}/t_{ij} , koje se koristi u trećem i četvrtom koraku, izračunava se jednom unapred pre evolucije za svaki računar, jer su vrednosti t_{ij} i p_{ij} konstantne tokom celog izvršavanja. Time se trošak sortiranja izbacuje iz unutrašnje petlje i amortizuje preko svih generacija, što je jedna od ključnih optimizacija koje omogućavaju primenu na velikim instancama.

2.8 Implementacija u programskim jezicima

Referentna implementacija algoritma razvijena je u programskom jeziku Python uz biblioteku NumPy [16], koja omogućava vektorizovane matrice operacije i znatno smanjuje overhead interpretera. Sve operacije popravke i evaluacije realizovane su kroz NumPy primitive nad celim matricama, dok je sortiranje servisa po profitabilnosti preneto u fazu pretpripreme pre evolucije. Ova vektorizacija je dovela do ubrzanja oko 5 puta u poređenju sa naivnom Python implementacijom.

Za velike instance, posebno onu dimenzija 1000×1000 , verzija u Pythonu prevazilazi prihvatljivo vreme izvršavanja. Zbog toga je razvijena i verzija u jeziku C++. Ona koristi tipove iz standardne biblioteke za vektorske strukture sa ručno linearizovanom dvodimenzionalnom indeksacijom radi bolje lokalnosti u kešu. Za generisanje slučajnih brojeva korišćen je Mersenne Twister generator. Izmereno ubrzanje u odnosu na verziju u Pythonu kreće se u opsegu od 20 do 50 puta. Za instancu 1000×1000 to znači razliku između nekoliko sati i nekoliko minuta po pokretanju.

Verzija u jeziku C++ pažljivo upravlja memorijom. Geni se čuvaju kao 16-bitni celi brojevi, što je dovoljno jer su vrednosti ograničene sa $\min(d_i, \lfloor T/t_{ij} \rfloor)$, a prepolovljuje memorijsku zauzetost u odnosu na standardni 32-bitni zapis. Hromozom je jedan neprekidan niz u memoriji (linearizovana matrica), a pri zameni generacija koristi se semantika premeštanja (eng. *move semantics*) umesto kopiranja, pa se skupa kopiranja vrše samo za elitne jedinke. Dodatno, hromozomi su posle popravke izrazito retki — na instanci 1000×1000 svega oko 12% gena je nenulto — pa kompresija memorije operativnog sistema značajno smanjuje stvarnu rezidentnu zauzetost u odnosu na nominalnu veličinu populacije.

2.9 Kriterijumi zaustavljanja

Pored zaustavljanja posle unapred zadatog broja generacija, u literaturi su uobičajeni i drugi kriterijumi. Prvi je zaustavljanje po isteku unapred zadatog vremena izvršavanja. Drugi je zaustavljanje po stagnaciji, gde se algoritam zaustavlja kada se najbolja vrednost ne poboljšava više od određenog broja generacija. Treći je zaustavljanje po raznovrsnosti, gde se algoritam zaustavlja kada se prosečna razlika između hromozoma u populaciji spusti ispod zadatog praga.

U sprovedenim eksperimentima korišćeno je zaustavljanje posle unapred zadatog broja generacija, jer ono obezbeđuje unapred poznato vreme izvršavanja i olakšava poređenje različitih konfiguracija algoritma. Pored toga, prati se i kriva kumulativnih maksimuma, što omogućuje retrospektivnu analizu da li je odabrani broj generacija bio dovoljan ili je potrebno povećati broj generacija.

2.10 Hardverska konfiguracija eksperimenata

Svi eksperimenti izvedeni su na istom računaru radi pravednog poređenja vremenskih merenja. Konfiguracija je MacBook Pro 16" iz 2021. godine, sa procesorom Apple M1 Pro, 16 GB RAM memorije i SSD diskom za skladištenje. Operativni sistem je macOS, a kompajler za verziju u C++ je g++ verzije 13 sa optimizacionim parametrima `-O3` i `-mcpu=apple-m1`. Verzija jezika Python je 3.11, a NumPy je verzije 1.26. Alat HiGHS korišćen je u verziji 1.7, dok je Gurobi pristupljen preko NEOS servera u verziji 11.

3 Rezultati

3.1 Postavka eksperimenta

Eksperimenti su sprovedeni na tri veličine problema, gde m označava broj servisa, a n broj računara. Problem male dimenzije ima vrednosti parametara $m = n = 10$. Vrednosti parametara za problem srednje dimenzije su $m = n = 100$, odnosno 100 servisa i 100 računara. Za problem velike dimenzije parametri imaju vrednosti $m = n = 1000$. U svim slučajevima maksimalno radno vreme računara iznosi $T = 2880$ minuta. Za svaku konfiguraciju algoritma izvedeno je više nezavisnih pokretanja, a izveštavaju se najbolja vrednost, medijana, prosek i standardna devijacija. Vreme izvršavanja se meri u sekundama.

Za svaku veličinu problema korišćena je po jedna slučajno generisana instanca. Vremena izvršavanja t_{ij} uzimaju vrednosti iz opsega od 4 do 17 minuta za malu instancu, odnosno od 5 do 24 minuta za srednju i veliku instancu. Zarade p_{ij} kreću se od 6 do 42 dinara za malu instancu, odnosno od 10 do 49 dinara za srednju i veliku. Zahtev d_i za malu instancu varira po servisima u opsegu od 200 do 1200 jedinica, dok je za srednju i veliku instancu fiksirana na 500 jedinica po servisu. Sve instance su vremenski zasićene: na optimalnom rešenju prosečna iskorišćenost radnog vremena računara premašuje 99,9%.

Sva izlazna stanja, krive konvergencije i sumarne statistike snimaju se u CSV i tekstualne datoteke za potrebe analize.

3.2 Problem male dimenzije

Problem male dimenzije, sa 10 servisa i 10 računara, korišćen je kao verifikaciona instanca. Cilj nije samo da se izmere performanse algoritama, već i da se proverí da li genetički algoritam uopšte može da dostigne optimum koji je u ovom slučaju lako proverljiv. Veličina linearnog programa je 100 promenljivih i 20 ograničenja, što je trivijalno za savremene alate za matematičku optimizaciju.

3.2.1 Rešenje alatom HiGHS

Optimalna vrednost dobijena alatom HiGHS iznosi 89847 dinara. Vreme izvršavanja je manje od jedne sekunde, čak i sa uključenim pretprocesiranjem. Gurobi preko NEOS servera potvrđuje istu vrednost. Time se utvrđuje referentna vrednost optimuma, prema kojoj se mere svi ostali algoritmi.

3.2.2 Rešenje genetičkim algoritmom

Genetički algoritam je testiran u više konfiguracija za ovu instancu, sa veličinama populacije $P \in \{100, 200, 500, 1000, 2000\}$, gde P označava veličinu populacije, G broj generacija, a E broj elitnih jedinki koje se direktno prenose u sledeću generaciju. Konkretno, korišćene su sledeće konfiguracije (oznake u formatu populacija-generacije-elitni):

- $P = 100, G = 1000, E = 3,$
- $P = 200, G = 500, E = 6,$
- $P = 500, G = 200, E = 15,$
- $P = 1000, G = 100, E = 30,$

- $P = 2000, G = 50, E = 50$.

Kao referentna metoda korišćena je slučajna pretraga sa 100000 evaluacija, u kojoj svako generisano rešenje prolazi kroz istu proceduru popravke kao i kod genetičkog algoritma. Svaka konfiguracija pokrenuta je 15 puta, a rezultati su usrednjeni. Gotovo sve konfiguracije postižu vrednost oko ili preko 98% optimuma u proseku (najslabija, (2000, 50), ostaje na 97,9%), što potvrđuje da je za ovu veličinu instance algoritam veoma uspešan.

3.2.3 Poređenje algoritama

Tabela 1 prikazuje rezultate različitih algoritama. U koloni *Procenat opt.* prikazan je odnos najbolje vrednosti koju algoritam pronalazi i optimalne vrednosti dobijene alatom HiGHS, izražen u procentima.

Algoritam	Najbolji	Medijana	Vreme [s]	Procenat opt.
HiGHS (ILP)	89847	–	< 1	100,0
Slučajna pretraga (100000 ev.)	75275	74394	0,2	83,8
GA (100, 1000)	88420	88080	0,03	98,4
GA (200, 500)	88636	88144	0,04	98,7
GA (500, 200)	88563	88415	0,04	98,6
GA (1000, 100)	88454	88317	0,05	98,5
GA (2000, 50)	88231	87997	0,05	98,2

Tabela 1: Rezultati za problem male dimenzije. ILP optimum: 89847.

Prosečna standardna devijacija genetičkog algoritma preko 15 pokretanja kreće se u opsegu 100–213 dinara, odnosno ispod 0,25% optimuma. Algoritam je veoma stabilan. Slučajna pretraga, čak i sa 100000 evaluacija, postiže samo 83,8% optimuma, dok genetički algoritam sa istim ukupnim brojem evaluacija (proizvod $P \cdot G$ za sve konfiguracije iz tabele iznosi 100000) prelazi 98%. Razlika ne potiče od broja evaluacija, već od genetičkog algoritma kao celine, koji kroz selekciju, rekombinaciju i mutaciju usmerava pretragu ka obećavajućim oblastima prostora rešenja.

Razlog za relativno visok kvalitet slučajne pretrage od 83,8% leži u činjenici da generisana rešenja prolaze kroz proceduru popravke, koja sadrži pohlepni korak popunjavanja. Time se još jednom potvrđuje da je dobar deo kvaliteta rešenja zasluga informisane konstrukcije, a ne same pretrage.

Što se tiče vremena izvršavanja, alat HiGHS i genetički algoritam su u istom redu veličine za ovu instancu (oba ispod jedne sekunde), pa za male instance nema razloga ne koristiti egzaktni pristup. Slučajna pretraga je nominalno najbrža, ali daje znatno lošije rezultate.

Interesantno je posmatrati odnos veličine populacije i broja generacija u dva različita slučaja. Tabela 2 prikazuje rezultate kada je broj generacija fiksiran na $G = 100$, a veličina populacije raste. Vidi se monotoni rast najboljeg i medijanskog rešenja sa rastom populacije: od 96,9% optimuma za $P = 100$ do 98,8% za $P = 2000$. Razlog je u tome da pri fiksnom broju generacija ukupan broj evaluacija raste sa populacijom, pa sa većom populacijom genetički algoritam nalazi bolje rešenje.

Konfiguracija	Najbolji	Medijana	Vreme [s]	Procenat opt.
GA (100, 100)	87028	86807	0,007	96,9
GA (200, 100)	87876	87498	0,013	97,8
GA (500, 100)	88328	88018	0,02	98,3
GA (1000, 100)	88454	88317	0,05	98,5
GA (1500, 100)	88595	88451	0,07	98,6
GA (2000, 100)	88746	88544	0,09	98,8

Tabela 2: Rezultati genetičkog algoritma za fiksni broj generacija $G = 100$ uz različite veličine populacije.

Suprotno tome, ako se fiksira ukupan broj evaluacija na 100000 (proizvod $P \cdot G$), trend se obrće. Kao što je vidljivo u tabeli 1, najbolji rezultati u proseku se postižu pri manjim populacijama sa više generacija. Konfiguracija (200, 500) ima najbolji pojedinačni rezultat 88636 dinara (98,7%), dok konfiguracija (2000, 50) pri istom broju evaluacija ostaje na 98,2%. Ovaj rezultat je značajan jer pokazuje da pri ograničenom broju evaluacija nije isplativo trošiti evaluacije na održavanje velike populacije ako se time žrtvuje broj generacija. Manja populacija sa više generacija dozvoljava više iteracija selekcije, ukrštanja i mutacije nad istim genetičkim materijalom, čime se intenzivnije eksploatišu obećavajuće oblasti prostora pretrage.

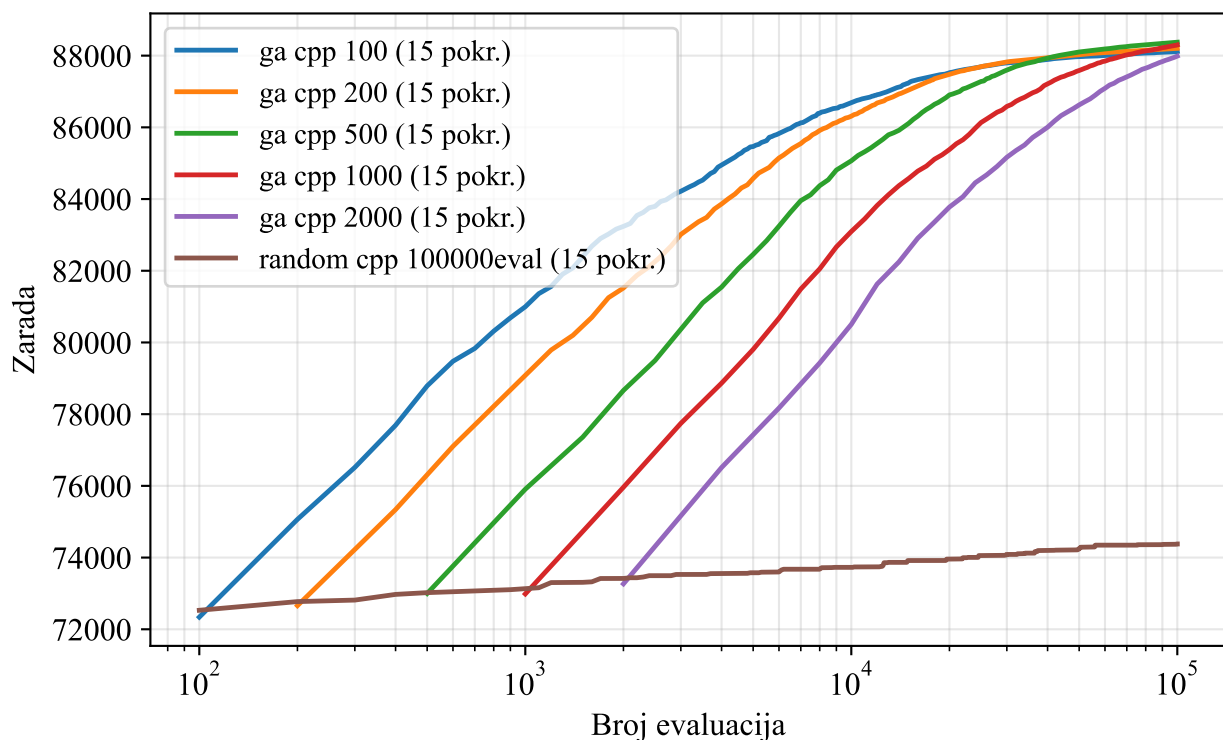
Ako se velikim populacijama dopusti više generacija (i samim tim veći broj evaluacija), rezultati nastavljaju da se popravljaju, kao što prikazuje tabela 3. Konfiguracija (2000, 500) sa milion evaluacija postiže 99,1% optimuma, dok (1000, 1000) dostiže 99,0%. Time se pokazuje da su veće populacije korisne samo ako im se obezbedi dovoljno generacija, jer u suprotnom dodatna raznolikost ne stiže da se iskoristi.

Konfiguracija	Najbolji	Medijana	Vreme [s]	Procenat opt.
GA (1000, 200)	88746	88473	0,08	98,8
GA (1000, 500)	88894	88697	0,17	98,9
GA (1000, 1000)	88926	88677	0,33	99,0
GA (1500, 100)	88595	88451	0,07	98,6
GA (2000, 100)	88746	88544	0,09	98,8
GA (2000, 500)	89010	88784	0,36	99,1

Tabela 3: Rezultati genetičkog algoritma za veće populacije uz povećan broj generacija.

3.2.4 Analiza kumulativnih maksimuma

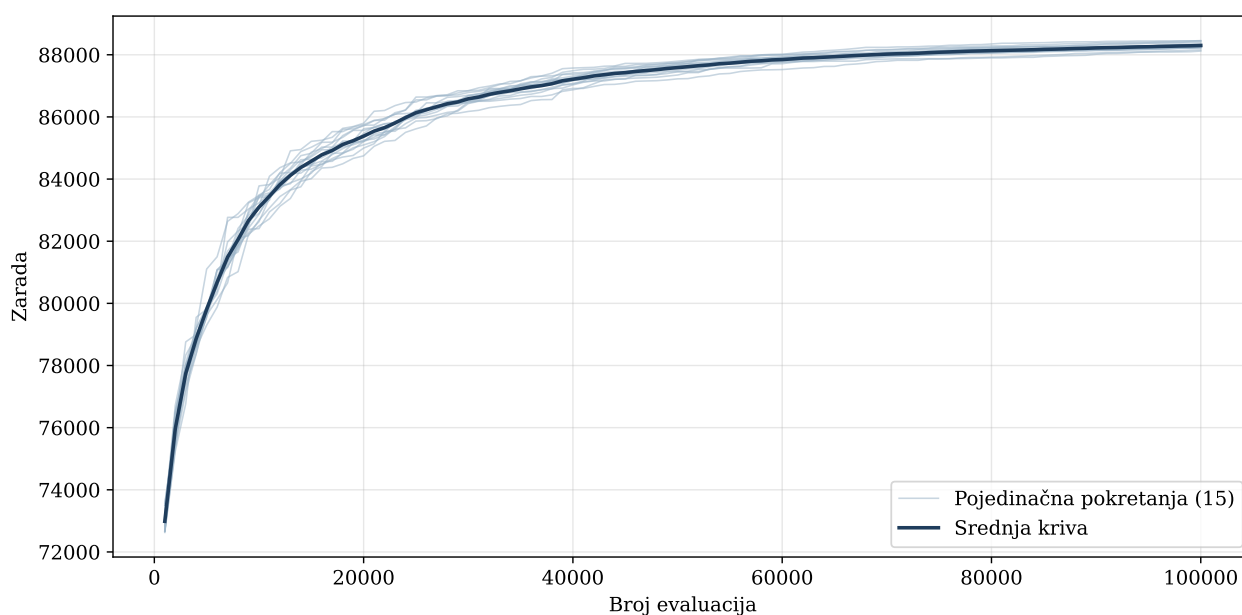
Slika 2 prikazuje uporedne krive srednjih kumulativnih maksimuma genetičkog algoritma za problem male dimenzije, usrednjene preko 15 pokretanja. Kriva u svakoj generaciji prikazuje najbolju do tada pronađenu vrednost funkcije cilja, koja je po samoj definiciji kumulativnog maksimuma monotonno neopadajuća. Poređenje pri većem broju evaluacija (oko milion), na kome se jasnije vide preseci krivih i zaravnjenja, dato je na slici 5.



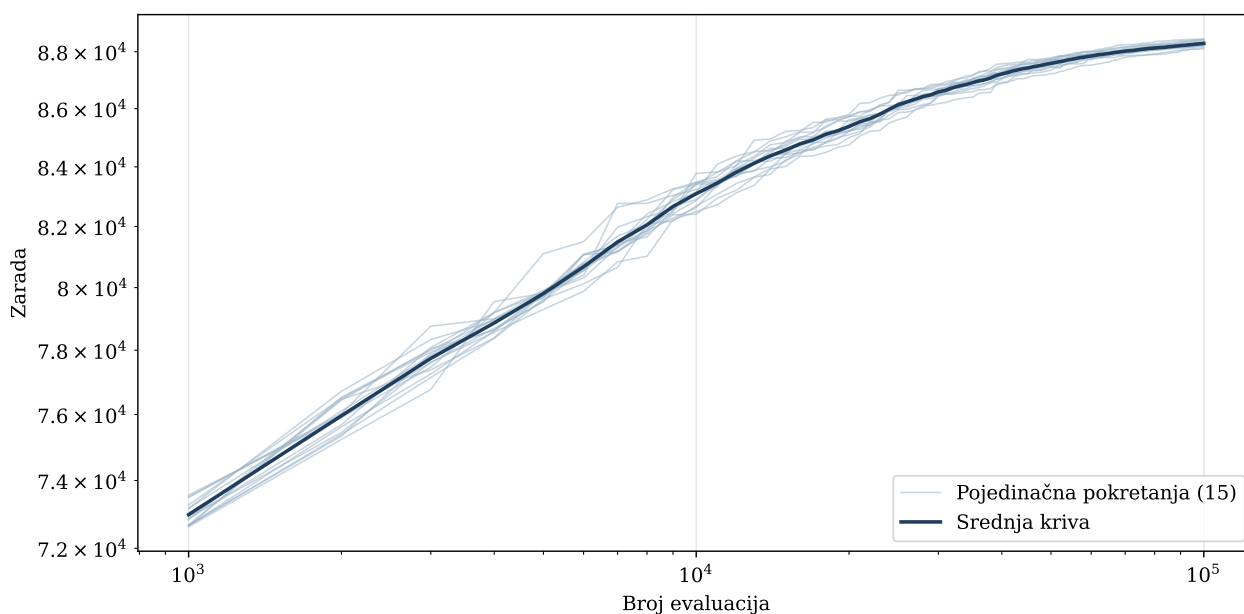
Slika 2: Poređenje krivih srednjih kumulativnih maksimuma za problem male dimenzije (15 pokretanja).

Diskusija krive konvergencije pokazuje karakterističan eksponencijalni profil. U prvih 5 do 10 generacija vrednost funkcije cilja raste izuzetno brzo, jer se kroz selekciju eliminišu najlošiji hromozomi iz početne populacije. Posle toga, rast se nastavlja sa nešto slabijim intenzitetom dok evolucija počinje da kombinuje gradeće blokove iz najboljih hromozoma. Konačno, kriva ulazi u dugu fazu finih poboljšanja, u kojoj svaka generacija donosi mali ali stabilan dobitak.

Slika 3 prikazuje krivu konvergencije u linearnoj skali, dok slika 4 prikazuje istu krivu u logaritamskoj skali po obe ose. Log-log skala pokazuje približno linearan trend u srednjem opsegu generacija, što ukazuje da brzina konvergencije sledi polinomski zakon, sa eksponentom manjim od jedinice. To je tipično ponašanje populacionih metaheuristika.

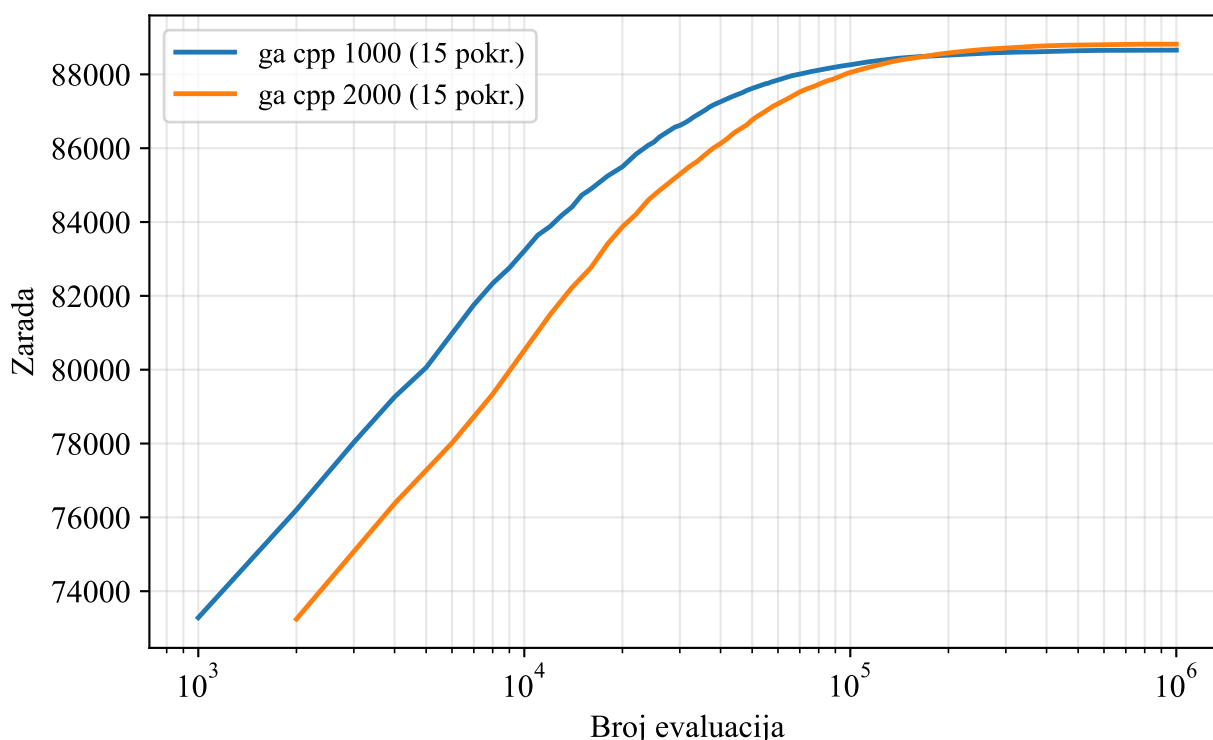


Slika 3: Kumulativni maksimum GA za problem male dimenzije, linearna skala (populacija 1000, 100 generacija, 30 elitnih; 15 pokretanja i srednja kriva).



Slika 4: Kumulativni maksimum GA za problem male dimenzije, log-log skala (15 pokretanja i srednja kriva).

Da bi se preciznije sagledao uticaj veličine populacije na kvalitet rešenja na kraju izvršavanja, slika 5 prikazuje poređenje dve konfiguracije sa približno istim brojem evaluacija (oko milion): populacija 1000 sa 1000 generacija (30 elitnih) i populacija 2000 sa 500 generacija (50 elitnih). Obe krive su usrednjene preko 15 pokretanja, a osa evaluacija je u logaritamskoj skali.



Slika 5: Poređenje krivih srednjih kumulativnih maksimuma za dve konfiguracije sa istim brojem evaluacija: populacija 1000 (1000 generacija, 30 elitnih) i populacija 2000 (500 generacija, 50 elitnih); 15 pokretanja po konfiguraciji.

Konfiguracija sa manjom populacijom u ranoj fazi pretrage ostvaruje brži rast jer manji broj jedinki znači više generacija, pa elitizam i selekcija agresivnije eksploatišu trenutno najbolje hromosome. Međutim, na kraju izvršavanja, veća populacija dostiže veću prosečnu vrednost funkcije cilja (88817 naspram 88657 u proseku, sa boljim najgorim slučajem od 88698 naspram 88338). Razlog je veća genetička raznolikost u svakoj generaciji, koja smanjuje verovatnoću prerane konvergencije ka lokalnom optimumu i omogućava da se u dugom roku otkriju kvalitetniji gradeći blokovi. Uopšteno, genetički algoritam sa manjom populacijom brže stiže do svog maksimuma, dok povećanje populacije podiže konačno dostignuti maksimum. To važi samo do određene veličine populacije: posle nje se dostignuti maksimumi međusobno veoma približavaju, pa čekanje na rezultat veće populacije prestaje da bude inženjerski opravdano.

3.3 Problem srednje dimenzije

Problem srednje dimenzije, sa 100 servisa i 100 računara, koristi se kao referentna instanca. Veličina linearnog programa je 10000 promenljivih i 200 ograničenja. Za ovu veličinu, ILP alat je još uvek vrlo efikasan, ali je vidljiv rast vremena izvršavanja u odnosu na malu instancu.

3.3.1 Rešenje alatom HiGHS

Optimalna vrednost dobijena alatom HiGHS iznosi 2023188 dinara, sa vremenom izvršavanja od 7,68 sekundi. Gurobi preko NEOS servera potvrđuje istu vrednost u kraćem vremenu od nekoliko

sekundi. Razlika u vremenu odražava razliku između open-source i komercijalnih alata, ali je HiGHS i dalje sasvim praktičan izbor za ovu veličinu.

3.3.2 Rešenje genetičkim algoritmom

Zbog značajno većeg prostora pretrage, za problem srednje dimenzije koristi se C++ implementacija genetičkog algoritma, koja omogućava rad sa znatno većim populacijama u prihvatljivom vremenu. Testirano je više konfiguracija, sa veličinama populacije $P \in \{5000, 10000, 100000, 150000, 200000\}$:

- $P = 5000, G = 100, E = 50$, turnir $k = 5, p_m = 0,30$,
- $P = 10000, G = 100, E = 500$, turnir $k = 10, p_m = 0,20$,
- $P = 100000, G = 100, E = 5000$, turnir $k = 15, p_m = 0,15$,
- $P = 150000, G = 200, E = 7500$, turnir $k = 15, p_m = 0,15$,
- $P = 200000, G = 200, E = 10000$, turnir $k = 15, p_m = 0,15$.

Verovatnoća ukrštanja je $p_c = 0,8$ u svim konfiguracijama. Broj pokretanja varira od 2 do 11 u zavisnosti od cene konfiguracije. Vreme izvršavanja jednog pokretanja kreće se od oko 16 sekundi za najmanju do oko 1780 sekundi (oko 30 minuta) za najveću konfiguraciju.

3.3.3 Poređenje algoritama

Tabela 4 prikazuje rezultate.

Algoritam	Najbolji	Prosek	Vreme [s]	Procenat opt.
HiGHS (ILP)	2023188	–	7,7	100,0
Slučajna pretraga (10^7 ev.)	1003308	–	1761	49,6
GA (5000, 100)	1984502	1981686	16	98,1
GA (10000, 100)	1989039	1985559	30	98,3
GA (100000, 100)	1996732	1995536	456	98,7
GA (150000, 200)	1997917	1997704	1383	98,8
GA (200000, 200)	1999707	1999503	1779	98,8

Tabela 4: Rezultati za problem srednje dimenzije. ILP optimum: 2023188. Vreme se odnosi na jedno pokretanje GA. Procenat opt. izračunat na osnovu *Najbolji*.

Pri ovoj veličini, alat HiGHS i dalje pronalazi optimum u svega nekoliko sekundi, pa je genetički algoritam korisniji kao kontrolni mehanizam koji pokazuje da se metaheuristika približava optimumu unutar 1,2%. Ako se posmatra odnos vremena i kvaliteta, HiGHS je u ovoj veličini superiorniji.

Posebno je upečatljiv rezultat slučajne pretrage. Sa deset miliona evaluacija, što je oko dvadeset puta više nego što troši najmanja konfiguracija genetičkog algoritma, slučajna pretraga dostiže svega 49,6% optimuma. Za razliku od male instance, gde je razlika između slučajne pretrage i genetičkog algoritma iznosila oko 15 procentnih poena, ovde ona prelazi 48 procentnih poena. Kriva slučajne pretrage na slici 6 praktično je ravna: najbolja vrednost pronađena među prvih sto uzoraka gotovo se ne poboljšava ni posle deset miliona uzoraka. Ovo je očekivano ponašanje neinformisane pretrage: verovatnoća da nezavisno izvučen uzorak nadmaši dotadašnji maksimum opada sa brojem uzoraka, pa

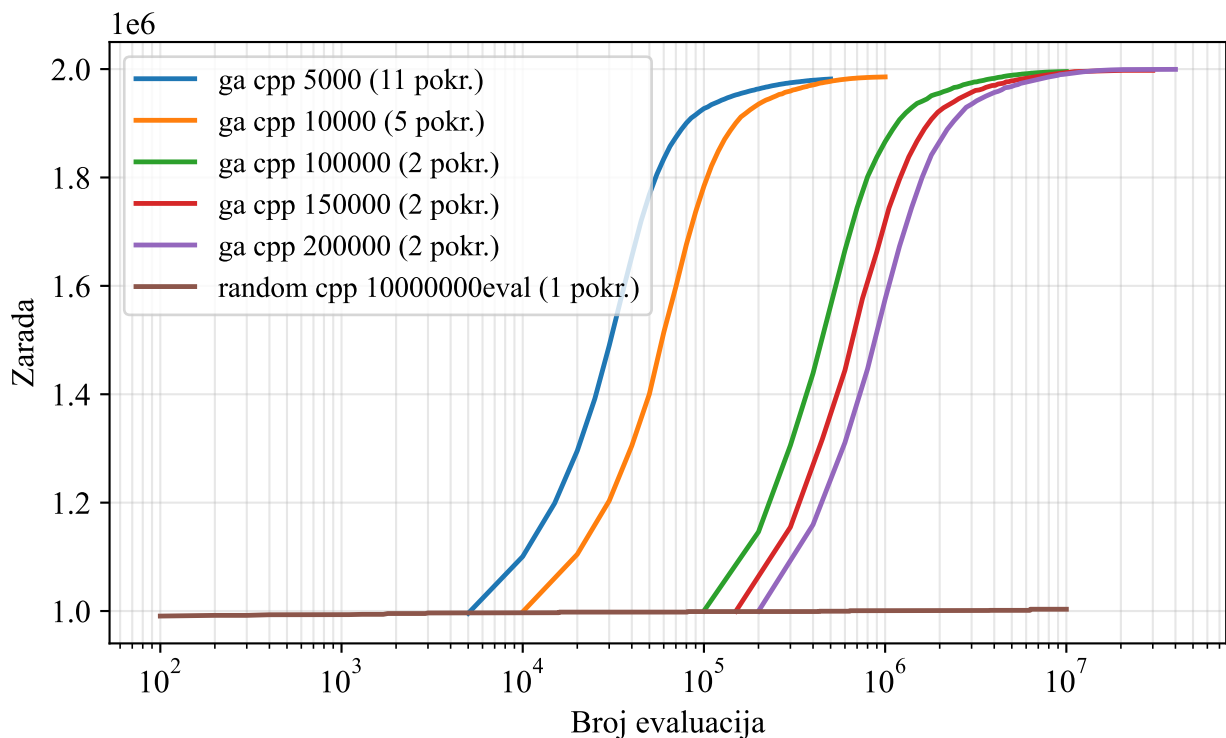
kvalitet raste tek logaritamski sporo. Sa rastom dimenzije problema ovaj efekat postaje još izraženiji, jer udeo visokokvalitetnih rešenja u prostoru pretrage eksponencijalno opada.

Vidljiv je monotoni rast kvaliteta sa rastom populacije: od 98,1% optimuma za $P = 5000$ do 98,8% za $P = 200000$, uz istovremeno smanjenje standardne devijacije sa 2534 na 204 dinara. Veće populacije daju i bolji i stabilniji rezultat, ali po ceni značajnog rasta vremena izvršavanja: prelazak sa $P = 100000$ na $P = 200000$ poboljšava prosečni rezultat za samo 0,2%, dok vreme raste skoro četiri puta.

Međutim, treba imati u vidu da HiGHS operiše na unapred poznatim ulazima, dok je genetički algoritam fleksibilniji kada se ulazi često menjaju ili kada postoji veliki broj scenarija koje treba evaluirati u kratkom roku. U industrijskoj praksi često se sreću scenariji u kojima se baza ulaznih instanci menja dnevno, pa se pretkalkulisana rešenja brzo zastareju i potrebna je metoda koja može da odgovori na nove ulaze za nekoliko sekundi.

3.3.4 Analiza kumulativnih maksimuma

Slika 6 prikazuje uporedne krive srednjih kumulativnih maksimuma genetičkog algoritma za problem srednje dimenzije, usrednjene preko 2 do 11 pokretanja, u zavisnosti od konfiguracije. Kao i kod male instance, kriva u svakoj generaciji prikazuje najbolju do tada pronađenu vrednost funkcije cilja, po definiciji monotono neopadajuću.

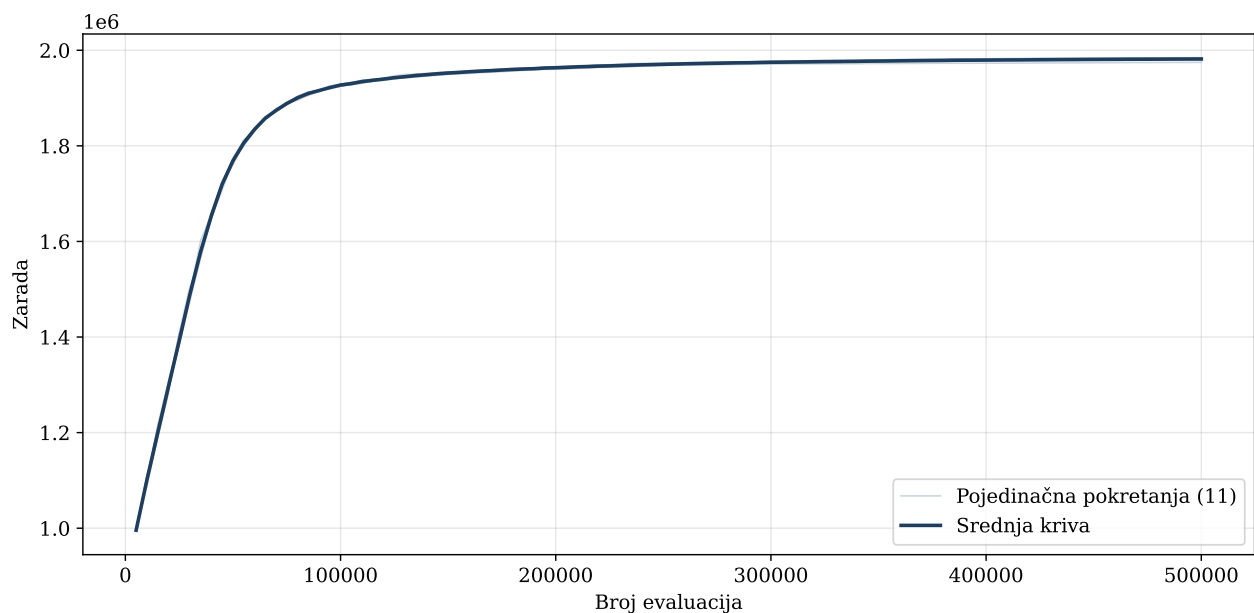


Slika 6: Poređenje krivih srednjih kumulativnih maksimuma za problem srednje dimenzije (2 do 11 pokretanja po konfiguraciji).

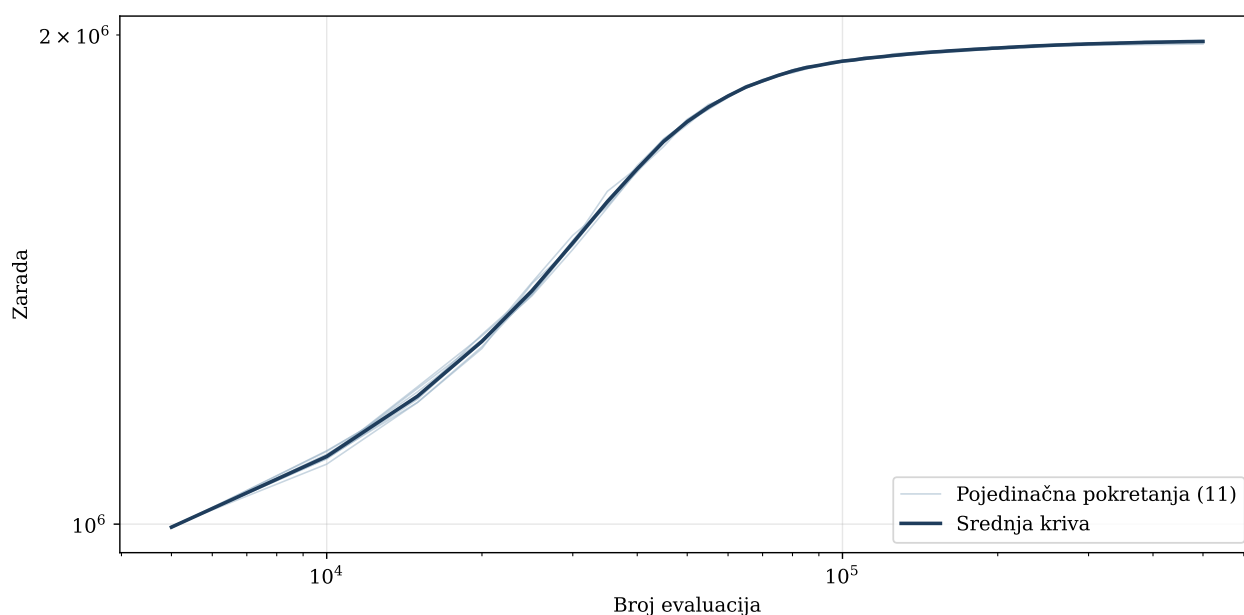
Diskusija krive konvergencije pokazuje isti karakterističan profil kao i kod male instance, ali sa bitno drugačijom početnom tačkom. Dok kod male instance procedura popravke već početnu populaciju vodi

do oko 80% optimuma, kod srednje instance najbolji hromozom početne populacije dostiže svega oko 49% optimuma (na primer, 995553 dinara za konfiguraciju sa $P = 5000$). Genetički algoritam zatim u prvih 10 do 20 generacija izuzetno brzo raste, jer se kroz selekciju eliminišu najlošiji hromozomi iz početne populacije, nastavlja sa nešto slabijim intenzitetom dok evolucija kombinuje gradeće blokove iz najboljih hromozoma, i konačno ulazi u dugu fazu finih poboljšanja, u kojoj svaka generacija donosi mali ali stabilan dobitak. Na ovoj instanci, dakle, evolutivna komponenta podiže kvalitet za skoro 50 procentnih poena, što pokazuje da se njen doprinos ne može svesti na fino doterivanje rešenja koje je proizvela popravka. Detaljnija analiza početnog kvaliteta po instancama data je u poglavlju 4.

Slika 7 prikazuje krivu konvergencije u linearnoj skali, dok slika 8 prikazuje istu krivu u logaritamskoj skali po obe ose. Log-log skala i ovde pokazuje približno linearan trend u srednjem opsegu generacija, što ukazuje da brzina konvergencije sledi polinomski zakon, sa eksponentom oko 0,4 (manjim nego kod male instance, što je očekivano zbog većeg prostora pretrage).



Slika 7: Kumulativni maksimum GA za problem srednje dimenzije, linearna skala (konfiguracija (5000, 100); 11 pokretanja i srednja kriva).



Slika 8: Kumulativni maksimum GA za problem srednje dimenzije, log-log skala (11 pokretanja i srednja kriva).

3.4 Problem velike dimenzije

Problem velike dimenzije, sa 1000 servisa i 1000 računara, predstavlja granicu primenljivosti egzaktnog pristupa u uslovima ograničenih resursa. Veličina linearnog programa je milion promenljivih i 2000 ograničenja. Memorijski zahtevi alata značajno rastu, kao i vreme izvršavanja.

3.4.1 Rešenje alatom HiGHS

Pri zadatom vremenskom limitu od 150 sekundi, HiGHS vraća rešenje sa vrednošću 20307561 dinara. Pri uklanjanju vremenskog limita, alat u oko 185 sekundi dokazuje da je upravo ta vrednost optimalna. Gurobi preko NEOS servera potvrđuje istu vrednost u kraćem vremenu, oko 90 sekundi.

3.4.2 Rešenje genetičkim algoritmom

Za problem velike dimenzije, Python implementacija postaje preskupa, pa se koristi C++ verzija. Testirane su četiri konfiguracije sa veličinama populacije $P \in \{8000, 10000, 15000, 20000\}$, brojem generacija $G = 100$, elitizmom od 80 do 200 jedinki i veličinom turnira od 10 do 20. Veliki P obezbeđuje raznolikost populacije, a manji G je posledica skupog izračunavanja funkcije cilja po hromozomu. Kao referentna konfiguracija uzima se $P = 15000$, $G = 100$, $E = 100$, sa 6 pokretanja; vreme izvršavanja jednog pokretanja iznosi oko 12300 sekundi, odnosno 3,4 sata. Pored toga, izvršena je i slučajna pretraga sa dva miliona evaluacija, što približno odgovara broju evaluacija referentne konfiguracije genetičkog algoritma ($P \cdot G = 1,5$ miliona).

3.4.3 Poređenje algoritama

Tabela 5 prikazuje rezultate.

Algoritam	Najbolji	Prosek	Vreme [s]	Procenat opt.
HiGHS (ILP)	20307561	–	185	100,0
Slučajna pretraga ($2 \cdot 10^6$ ev.)	17864312	–	38214	88,0
GA C++ (8000, 100)	19679643	19672354	6189	96,9
GA C++ (10000, 100)	19607996	19602755	7912	96,6
GA C++ (15000, 100)	19696956	19691496	12324	97,0
GA C++ (20000, 100)	19724152	19718844	15594	97,1

Tabela 5: Rezultati za problem velike dimenzije. ILP optimum: 20307561. Vreme se odnosi na jedno pokretanje. Procenat opt. izračunat na osnovu *Najbolji*.

Pri ovoj veličini HiGHS pronalazi bolje rešenje: genetički algoritam nalazi oko 3% lošije rešenje od optimuma. Vremena izvršavanja su značajno različita, oko dva reda veličine u korist HiGHS-a. Time se pokazuje da granica primenljivosti ILP-a nije apsolutna ni za ovu veličinu instance, već se izražava kroz cenu resursa. Genetički algoritam u ovoj situaciji nije zamena već dopuna, korisna u situacijama gde se proširenja problema teško izražavaju u ILP formulaciji, na primer kada zarada po jedinici servisa nelinearno zavisi od ukupnog broja izvršenih jedinica kroz količinske popuste ili zasićenje tržišta (odeljak 1.4).

Poređenje konfiguracija genetičkog algoritma pokazuje da kvalitet rešenja raste sa veličinom populacije, ali sa opadajućim prinosom: prosek raste sa 96,9% za $P = 8000$ na 97,1% za $P = 20000$, dok vreme izvršavanja raste približno linearno sa P , sa 1,7 na 4,3 sata po pokretanju. Odstupanje od monotonog trenda kod konfiguracije $P = 10000$ objašnjava se manjom veličinom turnira ($k = 10$ naspram $k = 15$ kod konfiguracije $P = 8000$), što daje slabiji selekcionni pritisak; ovo ilustruje da se veličina turnira mora skalirati zajedno sa populacijom.

Rezultat slučajne pretrage zaslužuje posebnu pažnju. Sa dva miliona evaluacija i ukupnim vremenom od 38214 sekundi (oko 10,6 sati, tri puta duže od referentne konfiguracije genetičkog algoritma), slučajna pretraga dostiže 17864312 dinara, odnosno 88,0% optimuma. Pri tome je najbolja vrednost među prvih 20000 uzoraka već iznosila 17854707 dinara: dodatnih 1,98 miliona evaluacija donelo je poboljšanje od svega 0,05%. Ovaj plato je posebno informativan zato što se najbolja jedinka početne populacije genetičkog algoritma, koja se generiše identičnim postupkom (slučajno rešenje propušteno kroz proceduru popravke), nalazi na praktično istoj vrednosti od oko 17,85 miliona dinara. Sav napredak genetičkog algoritma iznad 88% optimuma potiče, dakle, isključivo od evolutivnih operatora, a ne od pukog broja uzoraka.

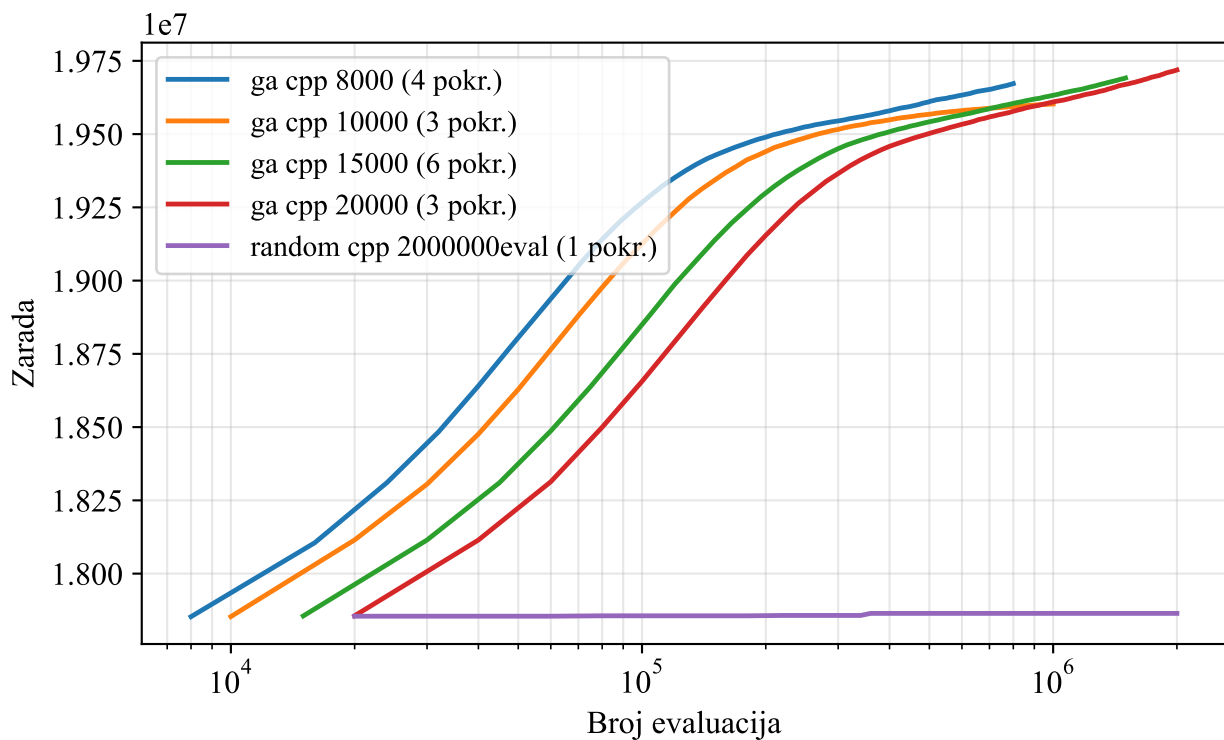
Zanimljivo je uporediti algoritme i pri jednakom raspoloživom vremenu. Jedna generacija referentne konfiguracije traje oko 123 sekunde, pa za vreme za koje HiGHS reši ceo problem do dokazane optimalnosti (oko 185 sekundi) genetički algoritam stigne da obradi tek jednu do dve generacije i nalazi se na oko 88–89% optimuma. Do svojih konačnih 97% genetički algoritam dolazi tek posle više sati. Ovo pokazuje da na ispitanoj instanci metaheuristika nema prednost ni u režimu ograničenog vremena; njena potencijalna prednost leži u fleksibilnosti modela i u scenarijima u kojima egzaktni pristup nije primenljiv.

Kod ove veličine instance dolaze do izražaja i efekti memorijske hijerarhije. C++ verzija čuva hromozom kao jedan neprekidan niz u memoriji (linearizovana matrica sa rasporedom po redovima), pa se izračunavanje funkcije cilja i procedura popravke svode na sekvencijalne prolaskе kroz memoriju, koji dobro koriste keš i hardversko pretpreuzimanje podataka. Predstavljanje hromozoma preko ugnežđenih struktura sa razbacanim alokacijama vodilo bi čestim promašajima keša i osetno dužem vremenu izvršavanja, što je dobro poznat efekat kod algoritama čija je brzina ograničena propusnošću

memorije. Pažnja prema niskonivouskim aspektima implementacije zato može biti jednako važna kao algoritamski izbori.

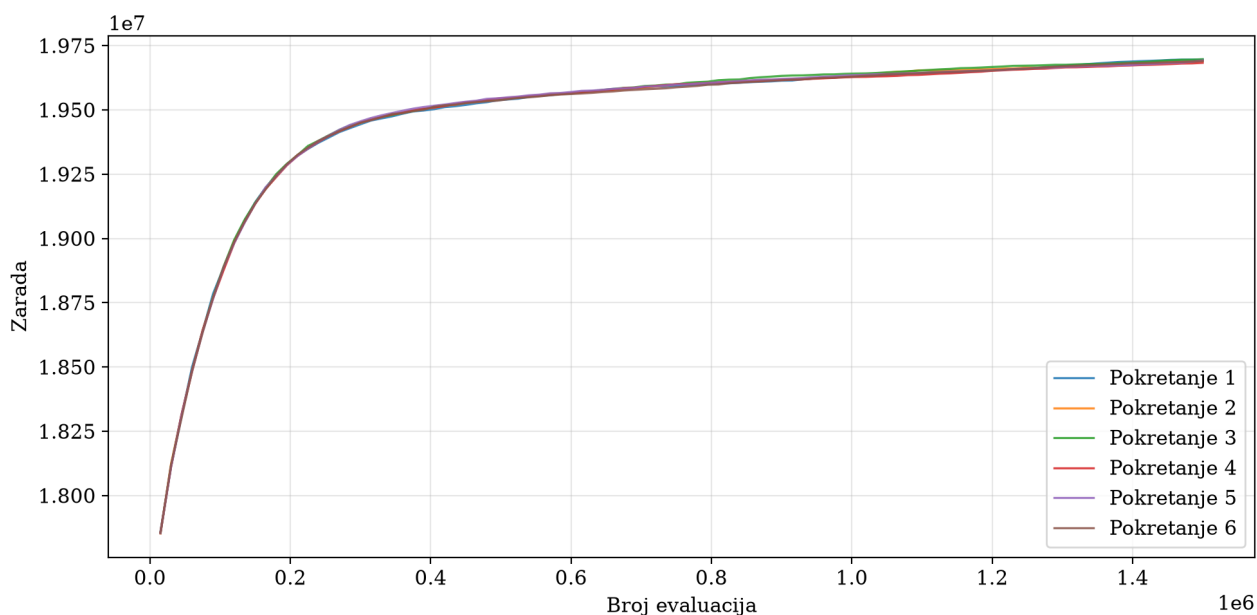
3.4.4 Analiza kumulativnih maksimuma

Slika 9 prikazuje uporedne krive srednjih kumulativnih maksimuma genetičkog algoritma za problem velike dimenzije, za nekoliko konfiguracija veličine populacije, usrednjene preko 3 do 6 pokretanja, u zavisnosti od konfiguracije. Kao i kod manjih instanci, kriva u svakoj generaciji prikazuje najbolju do tada pronađenu vrednost funkcije cilja, po definiciji monotonu neopadajuću.



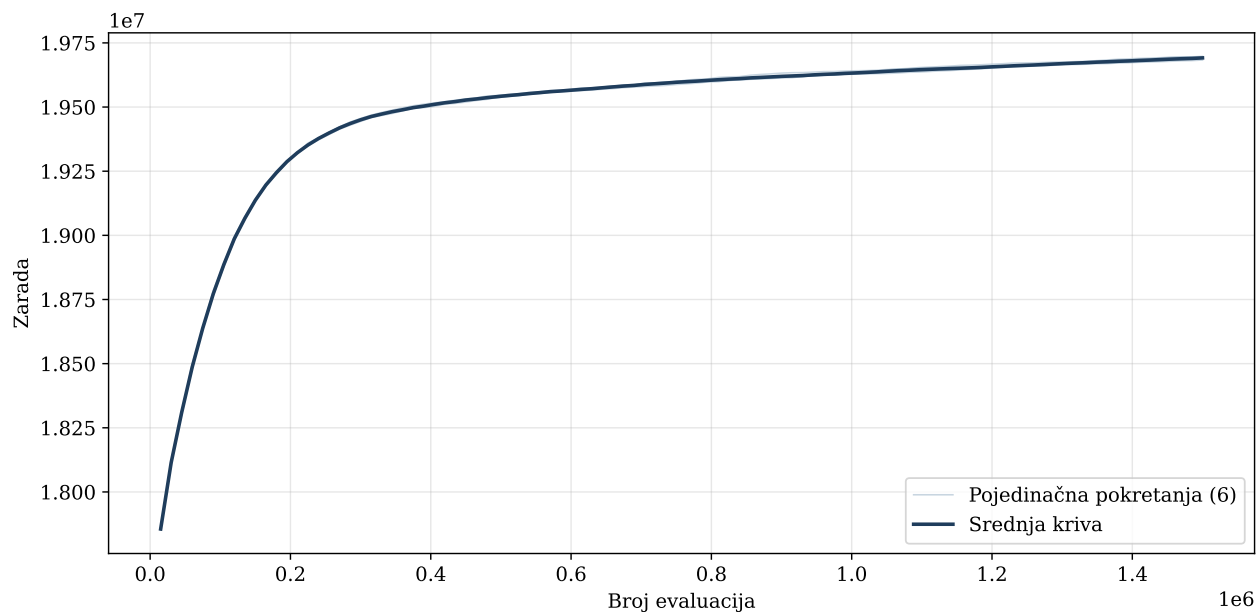
Slika 9: Poređenje krivih srednjih kumulativnih maksimuma za problem velike dimenzije (3 do 6 pokretanja po konfiguraciji).

Slika 10 prikazuje krive kumulativnih maksimuma za 6 pokretanja na problemu velike dimenzije. Vidi se da se sve krive grupišu u uskom opsegu, što ukazuje na stabilnost algoritma uprkos velikom broju promenljivih.

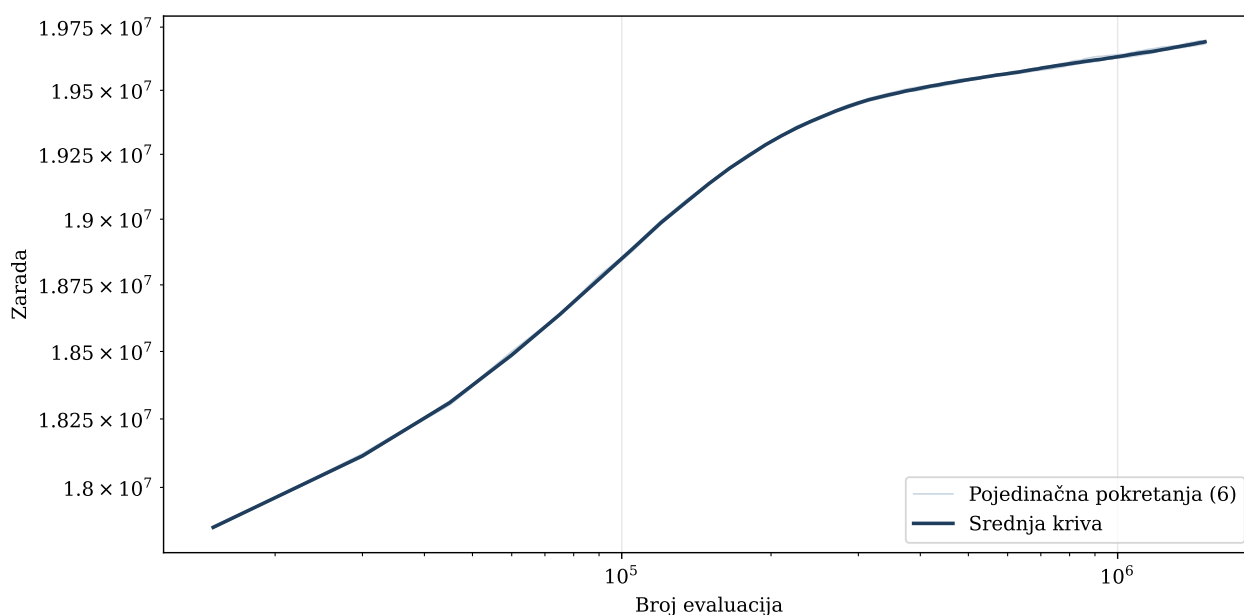


Slika 10: Kumulativni maksimumi za 6 pokretanja na problemu velike dimenzije.

Slike 11 i 12 prikazuju srednju krivu kumulativnog maksimuma u linearnoj i log-log skali.



Slika 11: Kumulativni maksimum GA za problem velike dimenzije, linearna skala (referentna konfiguracija (15000, 100); 6 pokretanja i srednja kriva).



Slika 12: Kumulativni maksimum GA za problem velike dimenzije, log-log skala (6 pokretanja i srednja kriva).

Diskusija krivih konvergencije za ovu instancu pokazuje sporiji relativni napredak po generaciji u poređenju sa manjim instancama. Razlog je u tome što je veličina prostora pretrage neuporedivo veća, pa svaka mutacija ili razmena gradećih blokova donosi srazmerno manji uticaj na ukupnu vrednost funkcije cilja. S druge strane, apsolutni dobici po generaciji u dinarima su znatno veći nego kod malih instanci, što odražava skalu problema.

Posmatrano u log-log skali, eksponent polinomske zavisnosti je oko 0,3, što je niže od onih izmerenih za manje instance. Time se potvrđuje opšta zakonitost da brzina relativne konvergencije opada sa rastom dimenzije problema, što je jedna od osnovnih teorijskih ograničenja stohastičkih metoda kod problema velikih dimenzija.

4 Analiza i diskusija rezultata

4.1 Uticaj hiperparametara

Podešavanje je sprovedeno prvenstveno nad dva ključna hiperparametra: veličinom populacije i brojem generacija. Umesto iscrpnog kombinovanja svih vrednosti, korišćene su unapred uparene konfiguracije. Na maloj instanci ispitano je fiksno brojeva od 100000 evaluacija sa populacijama od 100 do 2000 (od konfiguracije (100, 1000) do (2000, 50)), zatim rast populacije pri fiksnom broju generacija $G = 100$, i najzad uvećan broj evaluacija do milion. Na srednjoj instanci ispitane su populacije od 5000 do 200000 sa 100 do 200 generacija, a na velikoj populacije od 8000 do 20000 sa 100 generacija. Ovakav plan omogućava i poređenje pri jednakom broju evaluacija i procenu marginalne korisnosti dodatnih resursa.

Preostala dva hiperparametra skalirana su zajedno sa veličinom populacije. Broj elitnih hromozoma zadavan je kao mali udeo populacije: oko 3% kod male instance, između 1% i 5% kod srednje, a ispod 1% kod velike instance, gde već i takav udeo čini oko stotinu jedinki. Veličina turnira takođe je vezana za veličinu populacije i kretala se od 4–5 kod malih do 10–20 kod najvećih populacija, jer bi fiksni turnir kod velike populacije davao preslab selekcionni pritisak.

Uticaj veličine populacije najčistije se vidi u režimu fiksnog broja generacija na maloj instanci (tabela 2): rast populacije sa 100 na 500 podiže prosečan rezultat sa 96,4% na 98,0% optimuma, dok svako dalje udvostručavanje donosi još samo oko 0,2 do 0,3 procentna poena. Broj generacija pokazuje slično zasićenje: pri populaciji 1000 na maloj instanci, produžavanje sa 500 na 1000 generacija više ne donosi merljivo poboljšanje proseka (razlika je unutar statističkog šuma preko 15 pokretanja). Ista zakonitost opadajućih prinosa važi i na srednjoj instanci, gde prelazak sa $P = 100000$ na $P = 200000$ popravlja prosek za svega 0,2%, uz skoro četvostruko duže vreme. Veličina turnira oko 5 daje najbolje rezultate za male populacije, dok je kod velikih populacija potrebno podići turnir na 15–20 da bi selekcionni pritisak ostao dovoljan; premali turnir smanjuje selekcionni pritisak, a preveliki ga preuveličava i ubrzava konvergenciju, što povećava rizik od zaglavljivanja u lokalnom optimumu. Da nedovoljan turnir realno košta, pokazuje velika instanca: konfiguracija (10000, 100) sa turnirom $k = 10$ daje lošiji prosek od manje konfiguracije (8000, 100) sa turnirom $k = 15$ (tabela 5).

Uticaj stope mutacije ispitano je sistematski na maloj instanci, sa konfiguracijom $(P, G, E, k) = (1000, 100, 30, 5)$, mutacijom jednog gena i po 15 pokretanja za svaku od osam stopa mutacije između 0,05 i 1,00. Rezultati su prikazani u tabeli 6. Kvalitet monotonno raste sa stopom mutacije i ulazi u plato oko $p_m = 0,4$ do 0,6, bez degradacije čak i pri $p_m = 1$. Ovakvo ponašanje objašnjava se sadejstvom sa procedurom popravke i elitizmom: mutacija jednog gena je blaga perturbacija čije destruktivne efekte popravka odmah otklona i rešenje lokalno re-optimizuje, a elitne jedinke su u svakom slučaju zaštićene, pa se mutacija ponaša kao praktično besplatan izvor raznolikosti. Korišćena vrednost $p_m = 0,15$ je, prema tome, konzervativan izbor: na maloj instanci nešto više stope daju blago bolje rezultate. Treba, međutim, imati u vidu da ovaj zaključak važi za mutaciju jednog gena; kod C++ konfiguracija za srednju i veliku instancu, gde mutacija menja grupu od 50 gena, destruktivni efekat po događaju je veći, pa je umerena stopa opravdanija. Na manjim populacijama srednje instance stopa je podignuta na 0,20 do 0,30 kako bi se nadoknadila manja raznolikost populacije.

Stopa mutacije p_m	Prosek	Najbolji	Procenat opt. (prosek)
0,05	87982	88335	97,9
0,10	88191	88478	98,2
0,15	88283	88445	98,3
0,25	88406	88643	98,4
0,40	88492	88704	98,5
0,60	88573	88709	98,6
0,80	88545	88840	98,6
1,00	88569	88798	98,6

Tabela 6: Uticaj stope mutacije na maloj instanci; konfiguracija (1000, 100, 30), turnir $k = 5$, mutacija jednog gena, 15 pokretanja po stopi. ILP optimum: 89847.

Analizom logova svih pokretanja utvrđeno je i kada se injekcija raznolikosti zaista aktivira. Uslov od 100 uzastopnih generacija bez poboljšanja najbolje vrednosti ispunjava se samo u dugim pokretanjima na maloj instanci: u svih 15 pokretanja konfiguracije (1000, 1000), u 14 od 15 pokretanja konfiguracije (2000, 500) i u 12 od 15 pokretanja konfiguracije (100, 1000), dok se u pokretanjima sa 200 i manje generacija, kao i na srednjoj i velikoj instanci gde se najbolja vrednost pomera gotovo svake generacije, mehanizam nikada ne aktivira. U pokretanjima u kojima je injekcija okinula nije zabeleženo naknadno poboljšanje najbolje vrednosti: ubačene slučajne jedinke startuju daleko ispod elite (oko 80% naspram oko 99% optimuma), pa kroz turnirsku selekciju retko dolaze do reprodukcije. Mehanizam je zadržan kao osiguranje od prerane konvergencije u dugim pokretanjima, ali njegov doprinos u sprovedenim eksperimentima nije demonstriran, pa bi njegova sistematska evaluacija (na primer ablacionom studijom) bila korisna dopuna.

Posebno je zanimljiva interakcija između veličine populacije i broja generacija pri fiksnom ukupnom broju evaluacija. Pošto je ukupan broj evaluacija proporcionalan proizvodu $P \cdot G$, povećanje jednog parametra pri fiksnom broju evaluacija nužno smanjuje drugi. Eksperimenti pokazuju da, pri jednakom broju evaluacija, manje populacije sa više generacija po pravilu daju nešto bolje konačne vrednosti od velikih populacija sa malo generacija, jer veći broj generacija omogućava više ciklusa selekcije i rekombinacije nad istim genetičkim materijalom. Izuzetak nastaje kod velikih instanci, gde mala populacija ne sadrži dovoljno raznolikosti da pokrije ogroman prostor pretrage, pa je tada potrebno održati veću populaciju i pored manjeg broja generacija. Ova zavisnost objašnjava zašto se preporučena konfiguracija menja sa veličinom instance.

Treba istaći i da podešavanje hiperparametara nije nezavisno od procedure popravke. Na maloj i velikoj instanci popravka već sama po sebi dovodi najbolju jedinku početne populacije na oko 80%, odnosno 88% optimuma, pa se uloga evolutivnih operatora tamo svodi prvenstveno na doterivanje preostalih desetak procenata; na srednjoj instanci, međutim, početna populacija dostiže svega oko 49% optimuma, pa evolutivna komponenta nosi glavninu posla (detaljna analiza data je u odeljku 4.3). Rezultati su ipak na svim instancama relativno neosetljivi na umerene promene hiperparametara, što je poželjna osobina u praksi jer smanjuje potrebu za skupim podešavanjem za svaku novu instancu. Robusnost algoritma na izbor hiperparametara tako je delom posledica snage ugrađenih lokalnih heuristika, a delom stabilizujućeg dejstva populacione pretrage same po sebi.

4.2 Poređenje sa slučajnom pretragom

Direktno poređenje genetičkog algoritma sa slučajnom pretragom jasno pokazuje superiornost informisane pretrage, i to na svim ispitanim veličinama problema. Tabela 7 sumira poređenje. Važno je naglasiti da i slučajna pretraga koristi istu proceduru popravke sa pohlepnim popunjavanjem, pa merena razlika izražava isključivo doprinos evolutivnih operatora, a ne razliku u konstrukciji rešenja.

Instanca	SP: evaluacije	SP: % opt.	GA: evaluacije	GA: % opt.
10×10	10^5	83,8	10^5	98,7
100×100	10^7	49,6	$5 \cdot 10^5$	98,1
1000×1000	$2 \cdot 10^6$	88,0	$1,5 \cdot 10^6$	97,0

Tabela 7: Slučajna pretraga (SP) naspram genetičkog algoritma (GA) po instancama; procenat optimuma na osnovu najboljeg pokretanja. Za GA je navedena po jedna reprezentativna konfiguracija iz tabela 1, 4 i 5.

Za problem male dimenzije, pri identičnom broju od 100000 evaluacija, slučajna pretraga dostiže 83,8% optimuma, a genetički algoritam 98,7%; sa desetostruko većim brojem evaluacija genetički algoritam prelazi 99%. Za problem srednje dimenzije razlika postaje drastična: slučajna pretraga ni sa dvadeset puta više evaluacija od genetičkog algoritma ne prelazi polovinu optimuma. Za problem velike dimenzije slučajna pretraga dostiže 88% optimuma, ali gotovo ceo taj kvalitet potiče od procedure popravke: najbolja vrednost se posle prvih nekoliko desetina hiljada uzoraka praktično više ne pomera, dok genetički algoritam sa manjim brojem evaluacija i tri puta kraćim vremenom dostiže 97%.

Razlika u kvalitetu posledica je usmerenog istraživanja prostora pretrage kroz selekciju, ukrštanje i elitizam, dok slučajna pretraga ne koristi informacije iz prethodnih iteracija: verovatnoća da novi nezavisni uzorak nadmaši dotadašnji maksimum opada sa brojem uzoraka, pa njena kriva kumulativnog maksimuma raste tek logaritamski sporo. Posmatrano u apsolutnom iznosu na maloj instanci, u najboljem pokretanju slučajna pretraga ostavlja oko 14500 dinara neiskorišćeno, dok genetički algoritam dostiže razliku manju od hiljadu dinara; na velikoj instanci razlika između slučajne pretrage i genetičkog algoritma iznosi oko 1,8 miliona dinara po planiranom periodu. Time se opravdava veći algoritamski trud kod razvoja genetičkog algoritma, jer se taj trud direktno pretvara u značajno bolji ekonomski rezultat.

4.3 Kvalitet početne populacije i doprinos procedure popravke

Pošto i inicijalizacija populacije i slučajna pretraga koriste isti postupak (slučajno rešenje propušteno kroz proceduru popravke), kvalitet najbolje jedinice početne populacije neposredno meri koliki deo konačnog rezultata obezbeđuje sama popravka. Iz krivih kumulativnih maksimuma očitava se sledeće: na maloj instanci najbolja početna jedinka dostiže oko 81% optimuma (72673 od 89847 dinara za konfiguraciju (1000, 100)), na srednjoj svega oko 49% (995553 od 2023188 dinara), a na velikoj oko 88% (oko 17,85 od 20,31 miliona dinara). Doprinos evolutivne komponente je, prema tome, oko 18 procentnih poena na maloj, oko 50 na srednjoj i oko 9 na velikoj instanci.

Dodatna analiza popravljenih slučajnih rešenja objašnjava ovu na prvi pogled neobičnu nemonotonost. U sva tri slučaja popravka u potpunosti iskoristi raspoloživo radno vreme računara (preko 99,9%), pa razlika u kvalitetu ne potiče od neiskorišćenih resursa, već od profitabilnosti jedinica koje to vreme zauzimaju. Na srednjoj instanci proporcionalno smanjivanje redova u drugom koraku

popravke očuva strukturu slučajnog rešenja, pa vreme računara ostane zauzeto pretežno slabo profitabilnim dodelama, a pohlepno popunjavanje nema prostora da ih ispravi. Na velikoj instanci, gde se isti zahtev od 500 jedinica raspodeljuje preko hiljadu računara, celobrojno odsecanje pri proporcionalnom smanjivanju obriše veliku većinu gena (posle popravke je svega oko 12% gena nenulto), pa pohlepno popunjavanje gradi rešenje praktično iznova i samo dostiže kvalitet blizak pohlepnoj heuristici. Ova analiza pokazuje da se doprinos procedure popravke ne može uopštiti: on zavisi od odnosa dimenzija problema i strukture ograničenja, a evolutivna komponenta je na srednjoj instanci nosila glavninu ukupnog kvaliteta.

4.4 Analiza stabilnosti rezultata

Pored prikazanih agregatnih vrednosti, sprovedena je i osnovna statistička analiza rezultata. Za problem male dimenzije, raspodela vrednosti funkcije cilja preko 15 pokretanja genetičkog algoritma je usko koncentrisana oko medijane, sa interkvartilnim opsegom manjim od 200 dinara. Standardna greška proseka iznosi oko 38 dinara, što je manje od 0,05% optimuma.

Za problem srednje dimenzije, broj pokretanja smanjen je zbog dužeg vremena izvršavanja i varira od 11 kod najmanjih do svega 2 kod najvećih konfiguracija, ali je raspodela rezultata i dalje vrlo uska, sa standardnom devijacijom od najviše 0,13% najbolje vrednosti (2534 dinara za konfiguraciju (5000, 100), odnosno svega 204 dinara za (200000, 200)). Za problem velike dimenzije izvedeno je 6 pokretanja referentne konfiguracije, što je iz praktičnih razloga (svako pokretanje oko 3,4 sata) bio gornji limit u okviru raspoloživog vremena; standardna devijacija iznosi oko 4600 dinara. Standardna devijacija u apsolutnom iznosu raste sa dimenzijom problema, ali relativna devijacija u procentima optimuma čak opada: sa oko 0,15% na maloj, preko oko 0,1% na srednjoj, do oko 0,02% na velikoj instanci. Ovakvo ponašanje je očekivano, jer se sa rastom broja gena pojedinačne slučajne odluke usrednjavaju, pa je algoritam na velikim instancama srazmerno još stabilniji nego na malim.

4.5 Analiza vremenske složenosti

Vremenska složenost jedne generacije genetičkog algoritma može se razložiti na pojedinačne operatore. Izračunavanje funkcije cilja za svaki hromozom iznosi $O(mn)$, pa je ukupan trošak za celu populaciju $O(P \cdot mn)$, gde je P veličina populacije. Sortiranje populacije po vrednosti funkcije cilja pri elitizmu iznosi $O(P \log P)$. Turnirska selekcija sa veličinom turnira k iznosi $O(k)$ po pozivu, a sa P poziva ukupno $O(Pk)$. Ukrštanje i mutacija svako iznosi $O(mn)$ po hromozomu, pa ukupno $O(P \cdot mn)$. Procedura popravke u svom najskupljem koraku, popunjavanju, iznosi $O(mn \log m)$ ako se koristi sortirani niz servisa, a ovaj sortirani niz se može preračunati jednom pre evolucije.

Sumarno, vremenska složenost jedne generacije iznosi $O(P \cdot mn \log m)$. Ako se izvodi G generacija, ukupna složenost je $O(G \cdot P \cdot mn \log m)$. Za problem srednje dimenzije sa $P = 5000$ i $G = 100$, broj osnovnih operacija je reda veličine $3 \cdot 10^{10}$, što odgovara realno izmerenih oko 16 sekundi izvršavanja. Za problem velike dimenzije sa $P = 15000$ i $G = 100$, broj operacija je reda $1,5 \cdot 10^{13}$, što odgovara izmerenih oko 12300 sekundi u C++ implementaciji. Odnos izmerenih vremena prati odnos procenjenih brojeva operacija po redu veličine, uz nešto nižu propusnost na velikoj instanci usled pritiska na memorijsku hijerarhiju.

Pored vremenske, značajna je i prostorna složenost algoritma. U svakom trenutku u memoriji se čuva cela populacija od P hromozoma, pri čemu svaki hromozom zauzima $O(mn)$ celobrojnih vrednosti, pa je ukupna memorijska zauzetost populacije $O(P \cdot mn)$. Za problem velike dimenzije sa $P = 15000$ i $mn = 10^6$, to znači čuvanje reda $1,5 \cdot 10^{10}$ celobrojnih vrednosti, što bi pri standardna

četiri bajta po vrednosti iznosilo oko 60 GB i višestruko premašilo raspoloživu radnu memoriju. C++ implementacija zato koristi kompaktan 16-bitni zapis gena, čime se nominalna zauzetost prepolovljuje, a pri zameni generacija hromozomi se premeštaju umesto da se kopiraju. Presudnu ulogu, međutim, igra činjenica da su hromozomi posle popravke izrazito retki (na velikoj instanci svega oko 12% gena je nenulto), pa kompresija memorije operativnog sistema stranice ispunjene nulama efikasno sažima: izmerena rezidentna zauzetost tokom izvršavanja na velikoj instanci iznosila je oko 4 GB. Pomoćne strukture, poput sortiranog niza servisa po profitabilnosti, zauzimaju dodatnih $O(mn)$ prostora po računaru, ali se računaju jednom unapred i ne utiču na asimptotsko ponašanje po generaciji.

Odnos vremenske i prostorne složenosti ima direktne praktične posledice na izbor veličine populacije za velike instance. Pošto i vreme i memorija rastu linearno sa P , postoji prirodna gornja granica veličine populacije koju hardver može da podrži. Upravo zbog toga je za problem velike dimenzije izabrana umerena populacija od 15000 jedinki uz mali broj generacija, umesto velike populacije koja bi premašila memorijske resurse korišćene mašine.

4.6 Konvergentni profil i zaključci

Krive kumulativnih maksimuma u sva tri eksperimenta pokazuju karakteristični obrazac sa tri faze. Prva faza, traje prvih 5 do 10% generacija i odlikuje se brzim rastom vrednosti funkcije cilja zahvaljujući eliminaciji najlošijih hromozoma iz početne populacije kroz selekciju. Druga faza, traje narednih 20 do 30% generacija i odlikuje se umerenim rastom kroz rekombinaciju gradećih blokova iz najboljih hromozoma. Treća faza, traje preostalih 60 do 70% generacija i odlikuje se sporim ali stabilnim rastom kroz mutacije i lokalne popravke.

U log-log skali, trend kumulativnog maksimuma je približno linearan u srednjem opsegu generacija. EkspONENT polinomske zavisnosti opada sa rastom dimenzije problema. Za problem male dimenzije ekspONENT je oko 0,5, za problem srednje dimenzije oko 0,4, a za problem velike dimenzije oko 0,3. Time se kvantifikuje opšta zakonitost da brzina relativne konvergentije opada sa rastom dimenzije, što je jedno od osnovnih teorijskih ograničenja stohastičkih metaheuristika.

4.7 Sažeta diskusija rezultata

Sva tri eksperimenta zajedno daju nekoliko zaključaka. Najpre, alat HiGHS je u svim ispitanim veličinama pronašao optimum u prihvatljivom vremenu, što potvrđuje njegovu superiornost kao izbor za probleme do veličine reda hiljadu po hiljadu. Genetički algoritam dostiže preko 99% optimuma na maloj instanci (uz uvećan broj evaluacija), oko 98,8% na srednjoj i oko 97% na velikoj instanci, što ga čini praktično primenljivim u scenarijima sa fleksibilnim zahtevima.

Drugi zaključak tiče se raspodele zasluga između procedure popravke i evolutivnih operatora. Doprinos pohlepnih heuristika u popravci je značajan, ali zavisen od instance: početna populacija dostiže oko 81% optimuma na maloj, oko 49% na srednjoj i oko 88% na velikoj instanci, pa evolutivna komponenta na srednjoj instanci nosi glavninu ukupnog kvaliteta (odeljak 4.3). Poređenje sa slučajnom pretragom, koja koristi istu popravku a ipak drastično zaostaje, pokazuje da su i popravka i evolucija neophodne komponente. Treći zaključak je da je niskonivouska optimizacija implementacije (vektORIZACIJA, linearizovana indeksacija, kompaktan zapis gena) od ključne važnosti za primenu na velikim instancama.

Statistička analiza pokazuje da je raspodela rezultata genetičkog algoritma usko koncentrisana oko medijane. Za problem male dimenzije, standardna greška proseka iznosi oko 38 dinara, što je manje

od 0,05% optimuma. Time se zaključuje da su prosečne vrednosti preko više pokretanja statistički pouzdana mera kvaliteta.

5 Zaključak

5.1 Pregled doprinosa

U ovom radu razmotren je problem maksimizacije zarade pri raspodeli servisa na heterogeni skup računara, formulisana kao varijanta generalizovanog problema dodeljivanja sa celobrojnima odlukama. Pokazano je da problem pripada klasi NP-teških kombinatornih problema, ali da egzaktno rešavanje preko celobrojnog linearnog programiranja u praksi ostaje konkurentan pristup i za velike instance, uz odgovarajuće vreme i memorijske resurse. Rezultati pokazuju da i za problem velike dimenzije alat HiGHS daje bolje vrednosti funkcije cilja od genetičkog algoritma.

Kao odgovor na ograničenja egzaktnog pristupa, implementiran je genetički algoritam sa specijalizovanom procedurom popravke koja integriše pohlepne lokalne heuristike. Algoritam koristi turnirsku selekciju sa elitizmom i injekciju raznolikosti, a realizovan je u programskim jezicima Python i C++. Eksperimenti su sprovedeni na tri reprezentativne veličine instanci sa po više pokretanja po konfiguraciji.

Rezultati pokazuju da genetički algoritam dostiže preko 99% optimuma na maloj instanci, oko 98,8% na srednjoj i oko 97% na velikoj instanci, što je znatno bolje od slučajne pretrage sa uporedivim ili čak većim brojem evaluacija: slučajna pretraga dostiže 83,8% na maloj, svega 49,6% na srednjoj i 88,0% na velikoj instanci. Time se potvrđuje da je razvijena metaheuristika praktično primenljiva u scenarijima gde se zahteva fleksibilnost u proširenjima modela. Alat HiGHS je u svim ispitanim veličinama instance dao bolja konačna rešenja u kraćem vremenu, pa metaheuristika predstavlja dopunu, a ne zamenu.

Zanimljiv nalaz rada tiče se raspodele doprinosa između lokalnih heuristika i evolucije. Pohlepne heuristike ugrađene u proceduru popravke daju snažnu početnu tačku na maloj i velikoj instanci (oko 81%, odnosno 88% optimuma), dok na srednjoj instanci početna populacija dostiže svega oko 49% optimuma, pa tamo evolutivna komponenta nosi glavninu ukupnog kvaliteta. Doprinos popravke, dakle, zavisi od strukture instance i ne može se uzeti zdravo za gotovo, što ukazuje koliko je važno pažljivo dizajnirati lokalne operatore u kontekstu konkretnog problema.

5.2 Ograničenja predloženog pristupa

Iako predloženi genetički algoritam efikasno rešava problem, važno je razumeti njegova ograničenja kako bi se pravilno usmerila njegova primena.

1. Kvalitet rešenja zavisi od pažljivog podešavanja hiperparametara. Veličina populacije, broj generacija, stope ukrštanja i mutacije, veličina turnira i broj elitnih hromozoma moraju se prilagoditi konkretnoj veličini instance. Univerzalna konfiguracija ne postoji.
2. Genetički algoritam ne garantuje optimalnost. Za problem velike dimenzije ostalo je oko 3% razlike u odnosu na optimum. U scenarijima gde je optimalnost neophodna, na primer u finansijskim aplikacijama, genetički algoritam ne može biti zamena za egzaktnu metodu.
3. Vremenski zahtevi za velike instance i dalje su visoki. Verzija u C++ donosi značajno ubrzanje, ali za problem velike dimenzije jedno pokretanje traje oko 3,4 sata, što može biti prepreka u scenarijima gde su potrebni brzi odgovori.
4. Predloženi algoritam je dizajniran za statičke instance problema. Dinamičke varijante, u kojima se zahtevi, vremena izvršavanja ili dostupnost računara mogu menjati tokom planiranog perioda, nisu pokrivena ovim radom.

-
5. Pretpostavka o nezavisnosti servisa može biti narušena u realnim sistemima sa deljenim resursima.
 6. Model ne uključuje otkaze računara i nepouzdanost hardvera.
 7. Funkcija cilja je jednokriterijumska, dok realne primene često zahtevaju balansiranje više ciljeva (zarade, energetske potrošnje, pravičnosti).
 8. Ne postoji garancija konvergencije u praktičnom vremenskom okviru.
 9. Eksperimenti su sprovedeni na po jednoj slučajno generisanoj instanci po veličini problema. Višestruka pokretanja obezbeđuju statističku pouzdanost u odnosu na stohastičnost samog algoritma, ali ne i u odnosu na varijabilnost instanci; analiza početnog kvaliteta u odeljku 4.3 pokazuje da ponašanje algoritma osetno zavisi od strukture instance, pa bi za čvršće uopštavanje zaključaka bilo potrebno više instanci po veličini.

5.3 Pravci daljeg rada

Ovaj rad bi mogao da se nadogradi u više pravaca.

Prvi pravac je uvođenje adaptivnih stopa mutacije i ukrštanja koje se prilagođavaju trenutnoj raznolikosti populacije, čime bi se izbeglo ručno podešavanje hiperparametara.

Drugi pravac je uvođenje eksplicitne lokalne pretrage u glavnu evolutivnu petlju, na primer kroz razmene jedinica između računara po principu 2-opt operatora.

Treći pravac je korišćenje rešenja relaksiranog linearnog programa kao početna inicijalizacija za populaciju genetičkog algoritma. Pretpostavka je da bi se preostalih 3% razlike na velikim instancama moglo na taj način dodatno smanjiti.

Četvrti pravac je proširenje modela na dinamičke instance problema, sa inkrementalnim operatorima popravke i ažuriranjem populacije u realnom vremenu.

Peti pravac je primena tehnika mašinskog učenja za predikciju kvalitetnih početnih rešenja. Neuronska mreža bi se obučila nad istorijskim instancama i naučila bi da preslikava ulaze (matrice vremena i zarade, vektor zahteva) u dobre početne raspodele.

Šesti pravac je razvoj specijalizovanih ukrštajućih operatora koji uvažavaju strukturu problema, na primer razmena celih kolona (rasporeda po računarima) ili redova (raspored servisa preko računara).

Sedmi pravac je kombinacija sa Lagranžovom relaksacijom, gde bi se dualne informacije iz relaksacije koristile za usmeravanje pretrage.

Osmi pravac je proširenje na više ciljeva istovremeno, kroz formulacije kao što su NSGA-II i SPEA2, čime bi se omogućilo balansiranje zarade sa energetsom potrošnjom, ravnomernom raspodelom opterećenja i poštovanjem rokova.

Deveti pravac je integracija sa sistemima za podršku odlučivanju i interaktivnim alatima, u kojima donosilac odluke može nametnuti dodatna ograničenja ili preferencije tokom rada algoritma.

Deseti pravac je sistematska evaluacija na većem broju slučajno generisanih instanci po veličini problema, sa različitim raspodelama vremena, zarada i zahteva, čime bi se proverilo koliko su izmereni odnosi između procedure popravke i evolutivne komponente osetljivi na strukturu instance.

5.4 Praktične preporuke

Na osnovu sprovedenih eksperimenata, mogu se formulisati sledeće praktične preporuke. Za probleme male i srednje dimenzije, sa do nekoliko hiljada promenljivih, preporučuje se direktna upotreba

alata HiGHS, koji garantuje optimalnost u prihvatljivom vremenu. Za probleme velike dimenzije, sa milionima promenljivih, HiGHS je na ispitanoj instanci postigao najbolji rezultat i u najkraćem vremenu, pa genetički algoritam u C++ implementaciji postaje praktičan izbor tek kada memorijski zahtevi punog ILP modela premaše raspoloživi hardver, ili kada proširenja modela (nelinearnosti, dinamičke promene ulaza) nije moguće izraziti u ILP formulaciji.

Pored izbora algoritma, važno je obratiti pažnju na podešavanje hiperparametara. Verovatnoća ukrštanja 0,8 i verovatnoća mutacije 0,15 pokazale su se robusnim na svim ispitanim veličinama. Ostale vrednosti treba skalirati sa instancom: za male instance dovoljna je populacija oko 1000 sa oko 1000 generacija, turnirom oko 5 i elitizmom oko 3% populacije; za velike instance preporučuju se populacije od desetina hiljada jedinki uz stotinak generacija, turnir od 15 do 20 i elitizam ispod 5% populacije, pri čemu je pri ograničenom broju evaluacija po pravilu isplativije smanjiti populaciju u korist broja generacija. Manje izmene ovih vrednosti unutar uobičajenih raspona ne menjaju značajno kvalitet rešenja, što ukazuje na robusnost predloženog pristupa.

Literatura

- [1] Silvano Martello i Paolo Toth. *Knapsack Problems: Algorithms and Computer Implementations*. Chichester: John Wiley & Sons, 1990.
- [2] Michael Sipser. *Introduction to the Theory of Computation*. 3. izdanje. Cengage Learning, 2012.
- [3] Marshall L. Fisher i Ramchandran Jaikumar. „A generalized assignment heuristic for vehicle routing”. U: *Networks* 11.2 (1981.), str. 109–124.
- [4] P. C. Chu i J. E. Beasley. „A genetic algorithm for the generalised assignment problem”. U: *Computers & Operations Research* 24.1 (1997.), str. 17–23.
- [5] Dragan Olćan i Jovana Petrović. *Linearno programiranje i Dancigov simpleks algoritam*. Prezentacija sa predavanja iz predmeta Optimizacioni algoritmi u inženjerstvu, Univerzitet u Beogradu – Elektrotehnički fakultet. Školska 2025/26. godina. URL: <http://mtt.etf.rs/ms/osnovni.optimizacioni.algoritmi.htm>.
- [6] Laurence A. Wolsey. *Integer Programming*. 2. izdanje. John Wiley & Sons, 2020.
- [7] Qi Huangfu i J. A. J. Hall. „Parallelizing the dual revised simplex method”. U: *Mathematical Programming Computation* 10.1 (2018.), str. 119–142.
- [8] HiGHS Development Team. *HiGHS – High Performance Software for Linear Optimization*. URL: <https://highs.dev>.
- [9] Gurobi Optimization. *Gurobi Optimizer Reference Manual*. 2024. URL: <https://www.gurobi.com>.
- [10] Stuart Mitchell, Michael O’Sullivan i Iain Dunning. *PuLP: A linear programming toolkit for Python*. Tehn. izv. University of Auckland, 2011.
- [11] John H. Holland. *Adaptation in Natural and Artificial Systems*. Ann Arbor: University of Michigan Press, 1975.
- [12] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley, 1989.
- [13] A. E. Eiben i J. E. Smith. *Introduction to Evolutionary Computing*. 2. izdanje. Springer, 2015.
- [14] Zbigniew Michalewicz. *Genetic Algorithms + Data Structures = Evolution Programs*. 3. izdanje. Springer, 1996.
- [15] Dragan Olćan i Jovana Petrović. *Genetički algoritam*. Prezentacija sa predavanja iz predmeta Optimizacioni algoritmi u inženjerstvu, Univerzitet u Beogradu – Elektrotehnički fakultet. Školska 2025/26. godina. URL: <http://mtt.etf.rs/ms/osnovni.optimizacioni.algoritmi.htm>.
- [16] NumPy Developers. *NumPy: The fundamental package for scientific computing with Python*. URL: <https://numpy.org>.

Spisak skraćenica

CPLEX	IBM ILOG CPLEX Optimizer
CSV	Comma-Separated Values
GA	Genetički algoritam (Genetic Algorithm)
HiGHS	High-performance open-source software for linear optimization
ILP	Integer Linear Programming
NEOS	Network-Enabled Optimization System
NLP	Nonlinear Programming
NSGA-II	Non-dominated Sorting Genetic Algorithm II
SPEA2	Strength Pareto Evolutionary Algorithm 2
SSD	Solid-State Drive

Spisak slika

Slika 1.	Blok dijagram genetičkog algoritma sa procedurom popravke i elitizmom. . .	10
Slika 2.	Poređenje krivih srednjih kumulativnih maksimuma za problem male dimenzije (15 pokretanja).	16
Slika 3.	Kumulativni maksimum GA za problem male dimenzije, linearna skala (populacija 1000, 100 generacija, 30 elitnih; 15 pokretanja i srednja kriva). . . .	17
Slika 4.	Kumulativni maksimum GA za problem male dimenzije, log-log skala (15 pokretanja i srednja kriva).	17
Slika 5.	Poređenje krivih srednjih kumulativnih maksimuma za dve konfiguracije sa istim brojem evaluacija: populacija 1000 (1000 generacija, 30 elitnih) i populacija 2000 (500 generacija, 50 elitnih); 15 pokretanja po konfiguraciji.	18
Slika 6.	Poređenje krivih srednjih kumulativnih maksimuma za problem srednje dimenzije (2 do 11 pokretanja po konfiguraciji).	20
Slika 7.	Kumulativni maksimum GA za problem srednje dimenzije, linearna skala (konfiguracija (5000, 100); 11 pokretanja i srednja kriva).	21
Slika 8.	Kumulativni maksimum GA za problem srednje dimenzije, log-log skala (11 pokretanja i srednja kriva).	22
Slika 9.	Poređenje krivih srednjih kumulativnih maksimuma za problem velike dimenzije (3 do 6 pokretanja po konfiguraciji).	24
Slika 10.	Kumulativni maksimumi za 6 pokretanja na problemu velike dimenzije. . . .	25
Slika 11.	Kumulativni maksimum GA za problem velike dimenzije, linearna skala (referentna konfiguracija (15000, 100); 6 pokretanja i srednja kriva).	25
Slika 12.	Kumulativni maksimum GA za problem velike dimenzije, log-log skala (6 pokretanja i srednja kriva).	26

Spisak tabela

Tabela 1.	Rezultati za problem male dimenzije. ILP optimum: 89847.	14
Tabela 2.	Rezultati genetičkog algoritma za fiksni broj generacija $G = 100$ uz različite veličine populacije.	15
Tabela 3.	Rezultati genetičkog algoritma za veće populacije uz povećan broj generacija.	15
Tabela 4.	Rezultati za problem srednje dimenzije. ILP optimum: 2023188. Vreme se odnosi na jedno pokretanje GA. Procenat opt. izračunat na osnovu <i>Najbolji</i> .	19
Tabela 5.	Rezultati za problem velike dimenzije. ILP optimum: 20307561. Vreme se odnosi na jedno pokretanje. Procenat opt. izračunat na osnovu <i>Najbolji</i> . . .	23
Tabela 6.	Uticaj stope mutacije na maloj instanci; konfiguracija (1000, 100, 30), turnir $k = 5$, mutacija jednog gena, 15 pokretanja po stopi. ILP optimum: 89847.	28
Tabela 7.	Slučajna pretraga (SP) naspram genetičkog algoritma (GA) po instancama; procenat optimuma na osnovu najboljeg pokretanja. Za GA je navedena po jedna reprezentativna konfiguracija iz tabela 1, 4 i 5.	29